PYX TRACKER

Web-based Enterprise Operations Management System

An Information Technology Capstone Project

Presented to the Faculty of Information and Communication Technology

South East Asian Institute of Technology, Inc.

National Highway, Brgy. Crossing Ruber, Tupi, South Cotabato

In Partial Fulfillment of the Requirements for the Degree of

BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY

CAYUNDA, KLINT RUSSEL R.

DELOS SANTOS, BRENT R.

FIN, NOLI III M.

Adviser: [Name to be confirmed]

September 2026

TABLE OF CONTENTS

CHAPTER I 4
Background of the Study 4
Objectives of the Study 6
General Objective 6
Specific Objectives 7
Scope and Limitation of the Study 8
Scope of the Study 8
Limitation of the Study 9
Significance of the Study 10
IPO Model 12
Definition of Terms 13
CHAPTER II 15
Web-Based Inventory and Production Management for Food SMEs 15
Parcel Processing Barcode Scanning and Shipment Verification 16
Product Costing and Pricing Information Systems 17
Multi-Platform Order Processing and Waybill Management 18
Point of Sale Systems and Integrated Retail Transactions 19
Return-to-Sender Management and Reverse Logistics 21
Automation and Sustainable Development Alignment 22
Synthesis of the Literature and Research Gap 23
CHAPTER III 24
Environment 24
Software Engineering Technology 25
Build 1 Integrated Inventory Management System 26
Build 2 Product Costing and Pricing Calculator 26
Build 3 Multi-Platform Order Processing and Waybill Management 27
Build 4 Parcel Processing and Inventory Synchronization 27
Build 5 Point of Sale System 28
Build 6 Return-to-Sender Management 28
Planning and Conception Initiation Phase 30
Business Model Canvas 30
Gantt Chart 31
Functional Decomposition Diagram 32
Analysis and Design Phase 33
Use Case Diagrams 33
Use Case Matrix 35
Storyboard 38

Database Design 45
Entity Relationship Diagram 46
Data Dictionary 48
Network Design 69
Network Topology 70
Technology Stack Development 71
Software Specification 71
Hardware Specification 72
Program Specification 73
Testing and Evaluation Procedure 73
List of Modules 75
REFERENCES 76

CHAPTER I

INTRODUCTION

Background of the Study

This study presents Pyx Tracker: Web-based Enterprise Operations Management System, a centralized web application developed for Pyx Food Products. It connects inventory and production records with online order preparation, parcel documentation, product costing, retail sales, and Return-to-Sender management. The project provides authorized personnel with a common record of stock and transactions as goods move through production and selling channels. Enterprise operations management in this study refers to these defined functions rather than every administrative or accounting activity of the company.

Pyx Food Products is located at Zone 1, Prk. Vicente Fin, Brgy. Magsaysay, Polomolok, South Cotabato, Philippines. The proposal identifies honey, Paragis Tea, Black Rice Coffee, and Jar Cacao among the covered product lines. The business receives materials, prepares products, maintains finished goods, and serves online and walk-in customers. Honey passes through warehouse, production, and packaging stages; department products require monitoring of raw inputs and finished pouches or jars. Before integration, employees recorded inventory on paper and in disconnected spreadsheets. They manually organized orders from Shopee, the VIP website and Facebook channels, matched waybills

to order lists, recorded retail sales, estimated costs, and maintained return records. Information therefore passed between staff responsible for materials, production, customer service, fulfillment, and sales. This description is based on the proposal, which does not provide timed observations or transaction volumes.

The proposal identifies six problems. (1) Manual parcel verification, inventory deduction, and packing-material monitoring increase the risk of stock discrepancies, dispatch delays, and incomplete shipment documentation. (2) Paper records and disconnected spreadsheets permit erasures, inconsistent entries, and counting errors, making physical stock difficult to reconcile with recorded balances. (3) Unstructured production and packaging cost calculations weaken the basis for selling-price decisions. (4) Manually organizing multi-channel orders and matching waybills increases verification work and the possibility of incorrect parcel contents. (5) Manual retail recording delays transactions and their inventory updates. (6) Unstructured return handling makes it difficult to preserve return history and distinguish physically received sellable goods from damaged goods or cancelled orders. These concerns limit the company's ability to trace an item from receipt to sale or return. They are documented problems rather than quantified error rates or measured business losses.

Pyx Tracker addresses these concerns through six connected modules. Integrated Inventory Management records receipts, production stages, finished goods, and stock movements. Product Costing and Pricing maintains product

formulas separately from parcel recipes. Multi-Platform Order Processing and Waybill Management connects preparation, validation, stamping, and batch records. Parcel Processing and Inventory Synchronization maintains packing and related stock information, while the current Deliveries station separately captures before-shipment photos. The Point of Sale module records walk-in transactions, receipts, and stock changes. Return-to-Sender Management preserves case and condition records and confirms sellable stock restoration through reviewed barcode stock-in sessions. Role-based access supports the responsibilities of executives, HR personnel, production managers, customer service staff, and cashiers. The expected benefit is a consistent, traceable operational record; actual improvement must be demonstrated through documented tests and business-workflow comparisons.

Objectives of the Study

General Objective

This study aims to develop Pyx Tracker, a web-based enterprise operations management system for Pyx Food Products, to centralize inventory, fulfillment, costing, retail sales, and return records.

Specific Objectives

1. Develop an integrated inventory module that records raw materials, production movements, packaging materials, supplies, and finished-goods balances.

2. Implement a costing module that calculates product costs from configured formulas and maintains separate parcel-packaging cost recipes.

3. Develop a multi-platform order module that prepares order files, matches waybills, and produces stamped documents and finalized order batches.

4. Implement a parcel-processing module that maintains traceable packing records, stock-movement references, and before-shipment photo documentation.

5. Develop a POS module that records retail transactions, produces receipts, and deducts the selected inventory source.

6. Implement an RTS module that preserves return history and confirms sellable stock restoration through reviewed barcode stock-in sessions.

The module-specific tests in Chapter III evaluate these objectives through observable balances, cost calculations, generated documents, transaction records, and confirmed stock changes.

Scope and Limitation of the Study

Scope of the Study

The study covers a role-based web application for the internal operations of Pyx Food Products. Its users include executives, HR personnel, production managers, authorized customer service and operational staff, and POS cashiers. Customers benefit from the resulting processes but are not authenticated users of a public storefront. The six major modules are as follows.

- Integrated Inventory Management System — records raw materials, honey production stages, department finished goods, packaging materials, supplies, and stock movements.
- Product Costing and Pricing Calculator — maintains product formulas, material prices, parcel-packaging recipes, and selling prices under executive access.
- Multi-Platform Order Processing and Waybill Management — prepares supported order files, stamps waybills, generates master Excel files, and finalizes batches.
- Parcel Processing and Inventory Synchronization — maintains packing and parcel records with applicable stock movements and separate shipment-photo documentation.
- Point of Sale System — records walk-in sales, deducts the selected stock source, generates receipts, and preserves sales history.

- Return-to-Sender Management — records return cases, item condition, claim history, and reviewed barcode stock-in of physically received sellable goods.

Authentication, staff and supplier records, permissions, dashboards, settings, and activity records support these six modules. They are shared controls rather than additional independent research objectives. The same module names are used in Chapter III.

Limitation of the Study

The project is limited to the identified company's workflows and configured product catalogs. It is not a multi-tenant commercial service or a public shopping platform. File preparation and waybill processing support implemented formats; universal marketplace compatibility and continuous API synchronization with Shopee, Facebook, or couriers are not claimed.

Users need access to the application server through a working network and compatible browser. Scanning depends on readable codes and suitable hardware or a supported camera environment. Reliable records require correct starting balances, mappings, units, prices, and confirmation of physical movements. A saved photo is documentary evidence and does not establish that the correct items were packed or delivered.

The current Deliveries photo-saving workflow does not deduct product or packing-material stock and does not automatically mark a parcel as shipped. Stock

movements occur through their designated inventory, packing, sales, or return operations. An RTS case does not itself increase sellable inventory; this requires a confirmed barcode stock-in session. Configured production settings also affect raw-material deduction, so automatic deduction is not assumed for every stock-in action.

The costing module depends on maintained formulas and prices. It is not a complete general ledger, payroll system, statutory accounting package, or autonomous pricing authority. The reviewed implementation does not substantiate predictive forecasting, a generative AI assistant, or an AI pricing engine. Completed usability scores, performance benchmarks, and quantified business improvement require separate evaluation evidence.

Significance of the Study

For executives, the system provides a basis for reviewing stock, sales, costs, and exceptions through connected records. It retains executive control over prices, permissions, and corrections.

For HR personnel, production managers, warehouse personnel, and customer service staff, role-specific workflows provide a consistent place to record receipts, production, packing, order preparation, shipment evidence, and returns. The expected benefit is less duplicate encoding and clearer responsibility for transactions.

For cashiers and customers, integrated POS records support organized transactions and retrievable receipts. Customers are indirect beneficiaries of consistent fulfillment and return documentation rather than assumed users of the internal application.

For future researchers, the project provides a Philippine food-enterprise case combining manufacturing-related inventory, online fulfillment, retail sales, and reverse flows. Its contribution can be evaluated through transaction traceability and documented implementation limits.

IPO Model

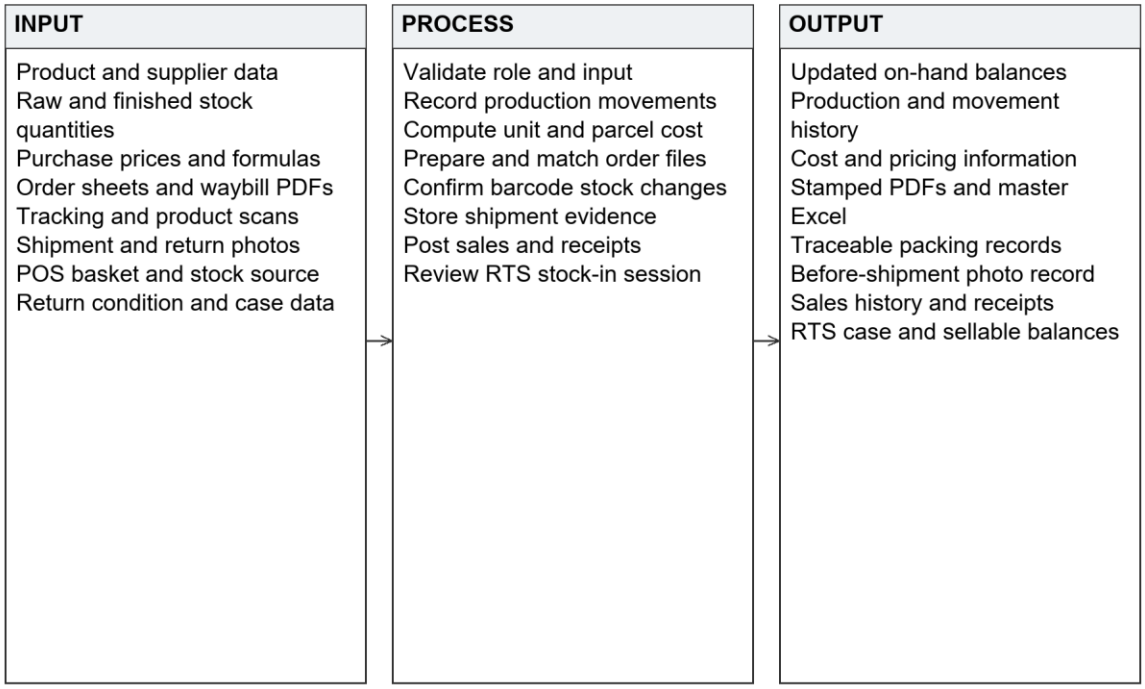


Figure 1.1 Input Process Output Model of Pyx Tracker

Inputs include product and supplier details, receipts, prices, order sheets, waybill PDFs, scans, photos, sales baskets, and return condition. The system validates access and input, records the relevant movement, calculates costs, prepares documents, and confirms reviewed stock changes. Outputs include balances, histories, stamped files, shipment evidence, receipts, and RTS records.

The model separates documentation from inventory posting. A tracking number and photo produce a shipment record through a save action; a scanned

sellable return produces an inventory increase after confirmation. Each output is therefore tied to the action that creates it.

Definition of Terms

The following terms are defined operationally for this study.

Barcode — A machine-readable identifier used to locate a product or tracking record. A permanent SKU code identifies a product type; a legacy unique code identifies an individual issued unit.

Before-shipment photo — An image stored against a tracking number. Saving it does not post inventory deductions.

Bill of materials — Configured quantities of raw inputs or packaging associated with a product or production operation.

Black Rice Coffee — The department product referred to as BRC.

Cost of goods sold — The product-cost figure called COGS in the system, maintained separately from outbound parcel-packaging cost.

Customer service representative — A CSR staff member who handles supported order or return operations within assigned permissions.

Finished goods — Configured sellable bottles, pouches, jars, or other units available after confirmed stock-in.

Inventory pool — A defined stock source such as normal finished goods, sellable RTS goods, or damaged returns.

Jar Cacao — The cacao product tracked as finished jars in department inventory.

Parcel recipe — Material quantities used to estimate packaging cost for a matching parcel composition or tier.

Point of Sale — The POS workflow for recording walk-in sales, related stock deductions, and receipts.

Return-to-Sender — The RTS workflow for documenting returned parcels and processing their goods according to condition and confirmed receipt.

Role-based access — Rules allowing users to perform operations appropriate to their assigned role and department.

Stock-keeping unit — A SKU representing a product type or variant rather than one individual item.

VIP — The internal label for website and Facebook orders processed through the VIP preparation workflow.

Waybill — A shipment document containing tracking and delivery information, matched or stamped during order processing.

CHAPTER II

REVIEW OF RELATED LITERATURE AND STUDIES

This chapter develops the six themes in the supplied proposal's literature review. Philippine studies provide local workflow comparisons, while foreign studies contribute evidence on costing, identification, and logistics. The review distinguishes prototypes, reported evaluations, and industry guidance. Its gaps refer to the reviewed works rather than claiming that no comparable system exists elsewhere.

Web-Based Inventory and Production Management for Food SMEs

Lorzano et al. (2022) linked POS transactions with a bill of materials for True North Empanada, addressing delays and discrepancies associated with manual consolidation. Laping et al. (2023) used design thinking and user evaluation to develop inventory-interface alternatives for a Marikina convenience store handling perishable goods. Ercilla et al. (2024) described a proposed inventory and POS system for Bida Mart with stock visibility and replenishment information. Together, these Philippine studies connect inventory problems with both transaction structure and staff needs.

The studies have different emphases: ingredient requirements, user-centered interface selection, and intended retail integration. A proposal should not be treated as proof of a mature deployment. These distinctions suggest that

inventory software needs a clear business-process model as well as usable screens.

Goedhart et al. (2023) extend the discussion to omni-channel retail, showing through numerical analysis that jointly considering replenishment and online allocation can improve inventory balance across stores. For Pyx Tracker, the relevant implication is shared stock visibility across channels, rather than adoption of their optimization model.

However, the reviewed studies do not document the same combination of honey production stages, department goods, packaging and supplies, packing, POS, and confirmed RTS stock-in. Pyx Tracker addresses this integration requirement while retaining physical counts and user confirmation as essential controls.

Parcel Processing Barcode Scanning and Shipment Verification

Chocholáč et al. (2022) compared RFID and barcode technologies in shipment consolidation through experiments and simulation. Their study concerns relative identification performance, not a general measured barcode-versus-manual improvement. Kamnardsiri et al. (2022) introduced barcode-detection datasets, including ParcelBar, and compared deep convolutional approaches. GS1 US (2026) discusses modern logistics data capture and two-dimensional identification.

These sources distinguish code detection, identifier interpretation, and the business action associated with a scan. A scan may locate a record without proving the contents or condition of a parcel. The local Bida Mart proposal identifies barcode integration as a future enhancement, not an established capability (Ercilla et al., 2024).

Pyx Tracker therefore treats scanning as an input mechanism supported by validation and confirmation. Tracking lookup supports documentary evidence; product scans support designated stock operations. Before-shipment photos provide a separate record for later return handling. The current Deliveries station preserves this distinction by saving photos without deducting stock.

However, the reviewed literature does not establish an evaluated Pyx-specific combination of packing records, product movements, material accountability, and shipment photos. The capstone must test these connections instead of transferring performance claims from RFID experiments or barcode-detection models.

Product Costing and Pricing Information Systems

Munifah et al. (2026) developed a web-based cost-of-goods-manufactured system for an Indonesian snack-food MSME using variable costing. Lorzano et al. (2022) connect material requirements with food-retail sales, while Divina et al. (2025) combine transactions, inventory, and analytics for Pastil sa Tabi. The latter

Philippine studies contribute related evidence on input and transaction integration; they are not equivalent parcel-costing systems.

Costing depends on both the calculation and its inputs. Quantity, purchase price, unit, and conversion yield must be interpreted consistently. A price per roll cannot represent the cost of one cut piece without a defined yield. Manufacturing cost and outbound parcel packaging also serve different purposes, so separate formulas make the calculation easier to review.

Pyx Tracker applies this separation through product formulas and parcel recipes. Executives maintain the values and selling prices. The resulting information supports a traceable decision but does not guarantee a commercially optimal price or profit. Accuracy remains conditional on complete and correct inputs.

However, the reviewed works do not document Pyx's exact connection among formulas, material yields, parcel tiers, and retail prices. The project addresses that integration requirement and checks calculations against known examples rather than treating automation itself as proof of accuracy.

Multi-Platform Order Processing and Waybill Management

Goedhart et al. (2023) show that online fulfillment and inventory decisions interact across time and locations. Nanayakkara et al. (2022) address e-commerce returns through clustering, circular-network design, and optimization. Divina et al.

(2025) illustrate transaction and inventory integration in a Philippine food enterprise. Although their settings differ, these works support examining how information remains connected through forward and reverse flows.

The Pyx proposal describes a file-oriented problem: staff organize supported channel orders and match them to waybills. This differs from selecting a fulfillment store or designing a regional returns network. The appropriate response is a controlled preparation workflow that permits review of names, items, variants, and tracking identifiers before parcel records are finalized.

The preparation tools and stamper perform distinct operations. VIP or SPX preparation produces an intermediate file; the PDF and Excel stamper is a separate step. Explicit stages help staff distinguish prepared input from generated output and finalized records. Duplicate-tracking validation also helps identify conflicting records before they are appended.

However, the reviewed studies do not document the same VIP and SPX preparation, PDF matching, master-file generation, and batch-finalization sequence. The project evaluates this organization-specific workflow using representative source files and expected output records.

Point of Sale Systems and Integrated Retail Transactions

Ercilla et al. (2024) present stock visibility and reporting as benefits intended by an integrated POS proposal. Lorzano et al. (2022) connect retail sales with

ingredient inventory. Divina et al. (2025) report user and technical evaluation of a POS and inventory system for Pastil sa Tabi, including favorable assessments and remaining performance and reliability concerns. This local evidence supports integration while showing that evaluation must extend beyond the presence of a checkout screen.

For Pyx Tracker, a sale must maintain consistency among items, quantities, prices, stock source, receipt, and ending balance. An RTS item must be distinguished from normal stock. Authentication and approval rules also affect a cashier's permitted actions.

Checkout is therefore assessed as a coordinated transaction. A test compares initial stock, purchased quantity, receipt total, and ending stock. Repeated or interrupted requests are examined for duplicate effects. These are evaluation requirements for this project rather than results already demonstrated by the proposal.

However, the reviewed works do not demonstrate the full Pyx combination of PIN access, department catalogs, selected normal or RTS stock, and wider production and fulfillment records. This capstone evaluates that combination within an operations platform.

Return-to-Sender Management and Reverse Logistics

Nanayakkara et al. (2022) develop a circular returns framework validated in an electrical and electronic goods case. Bhatti and Bauirzhanovna (2023), using interviews with five experts, discuss intelligent inventory systems in Saudi manufacturing reverse logistics. Chocholáč et al. (2022) contribute complementary identification research at shipment-consolidation points. These foreign studies support structured return processes while differing from the food-enterprise setting.

A return record is not the same as usable stock. A case can contain photos, claims, and reusable or damaged item lines, while sellable inventory should represent goods physically received and accepted into the pool. A cancellation, expected return, and received sellable return are different events.

Pyx Tracker separates case recording from a reviewed barcode stock-in session. Scans build a draft; discarding the draft leaves stock unchanged; confirmation posts reviewed quantities. Damaged goods remain distinct. This establishes a testable requirement: entering a case must not itself create sellable inventory.

However, the reviewed sources do not provide a Philippine implementation study of this exact case-to-confirmed-stock process connected with Pyx's sales

and packing workflows. That local evidence must come from the project's recorded return scenarios and inventory checks.

Automation and Sustainable Development Alignment

The proposal aligns the project with SDGs 8, 9, and 12. The United Nations identifies productivity and technological upgrading under Goal 8, industrial innovation under Goal 9, and resource efficiency and waste reduction under Goal 12 (United Nations Department of Economic and Social Affairs [UN DESA], n.d.-a, n.d.-b, n.d.-c). The project's intended contribution is operational: less repeated encoding, connected records, and greater visibility of materials and returns. It does not claim measured achievement of these targets.

AI methods in the barcode and intelligent-inventory literature do not establish that Pyx Tracker implements those methods. The supplied proposal and reviewed application support document processing, scanning, and configured business rules. This manuscript makes no claim of a trained forecasting model, autonomous decision engine, or generative AI feature.

However, an SDG alignment statement cannot replace evidence of reduced waste or improved productivity. Later impact claims should identify the measure, period, and business comparison used.

Synthesis of the Literature and Research Gap

The reviewed works share a concern with connections among material requirements, sales, inventory, orders, identification, and returns. Philippine studies offer local comparisons, while foreign logistics and costing studies explain additional dependencies. Their results remain specific to their settings and do not establish Pyx Tracker's effectiveness in advance.

The gap addressed here is the need for a coherent Pyx Food Products workflow connecting production-aware inventory, separate product and parcel costing, file-based orders, traceable shipment documentation, retail sales, and confirmed RTS stock-in. The contribution is integration and evaluation under the company's transaction and role rules, rather than a claim that every individual feature is novel.

Therefore, this study addresses these limitations by developing Pyx Tracker: Web-based Enterprise Operations Management System with six connected modules and a shared operational record. Chapter III explains the methodology, interactions, data structures, and verification procedure.

CHAPTER III

METHODOLOGY

This chapter describes the development approach, operating environment, functional design, data structure, and verification method for Pyx Tracker. It follows the supplied outline's environment, engineering technology, incremental builds, planning artifacts, analysis and design, database documentation, network design, specifications, and modules. Deployment at Pyx Food Products is acknowledged from the project team's account; dates, installed hardware, respondent counts, and measured results require supporting records.

Environment

The study is situated at Pyx Food Products, Zone 1, Prk. Vicente Fin, Brgy. Magsaysay, Polomolok, South Cotabato, Philippines. Its setting includes material receiving, production, online orders, packing, shipment documentation, retail sales, and return handling. Executives, HR personnel, production managers, customer service staff, and authorized warehouse and sales personnel participate in these workflows.

The proposal supplies the business scope. Reviewed project files and the local database schema supply implementation evidence for this chapter. These do not establish that every local change is installed identically on the company's

server. Identifying the release used during evaluation is therefore part of deployment verification.

Software Engineering Technology

The study uses the Agile Software Development Model identified in the proposal. Agile principles emphasize working software, collaboration, responsiveness to requirements, and regular reflection (Beck et al., 2001). The methodology organizes work into six increments, each covering design and development, testing, and implementation or refinement. Shared authentication and data definitions support every increment.

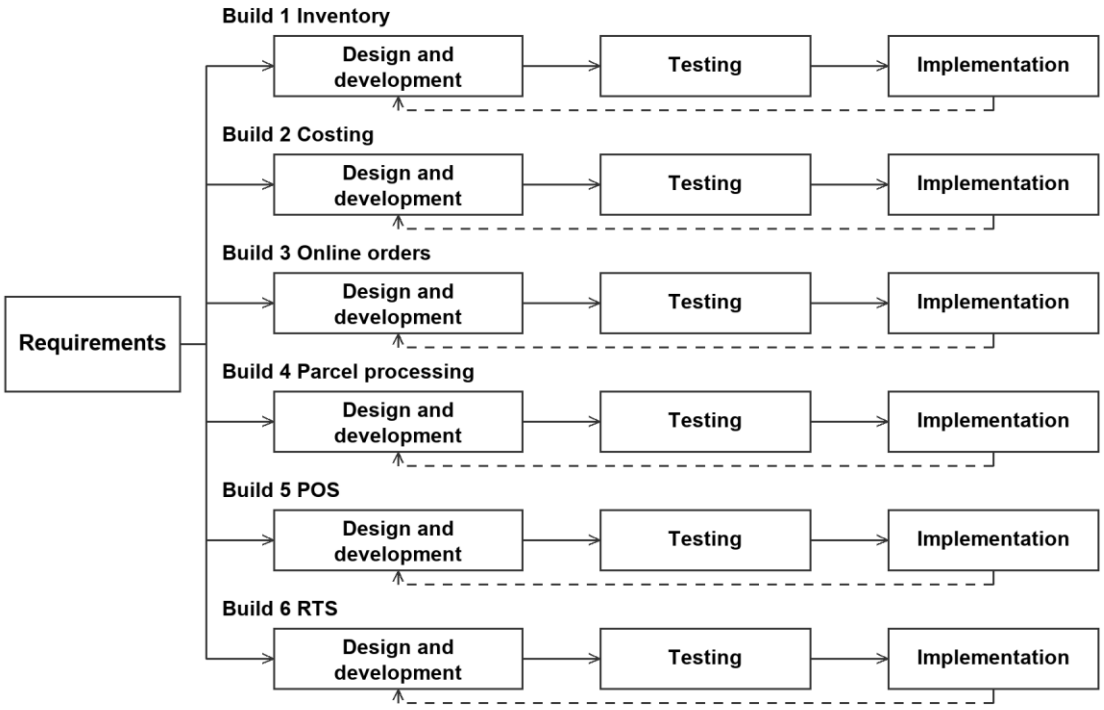


Figure 3.1 Incremental Agile Software Development Model

The diagram follows the sample's requirements-to-build structure, expanded to the six modules. Return arrows indicate feedback and revision. This sequence organizes the methodology and does not represent verified historical dates.

Build 1 Integrated Inventory Management System

Design and development define material categories, suppliers, products, measurement units, honey stage movements, and department finished goods. Separate stock structures preserve normal and returned sellable balances. Role rules determine who records receipts, performs production actions, and approves corrections.

Testing compares initial balances, recorded movements, and expected ending balances. Scenarios cover honey stages, department stock-in, configured raw deduction, repeated scans, and cancellation before confirmation. Raw kilograms must not be combined directly with finished pouches or bottles. Implementation review demonstrates receipt-to-finished-stock traceability and records the deduction settings used during evaluation.

Build 2 Product Costing and Pricing Calculator

Design and development establish formulas, material prices, yield conversions, custom costs, and parcel recipes. Manufacturing cost remains

separate from fulfillment packaging. Executive permissions govern costing and selling-price changes.

Testing uses known inputs to verify line calculations and totals, including a roll cost converted to a usable-piece cost, a custom labor line, and a parcel matching one tier. It also checks unauthorized access and missing or inactive recipes. Implementation review records formula inputs and outputs without claiming that cost computation guarantees profit.

Build 3 Multi-Platform Order Processing and Waybill Management

Design and development organize supported VIP and SPX inputs into a common preparation workflow. Users review names, items, and variants, then operate the separate PDF and Excel stamper. Finalized batches provide master-file and parcel records.

Testing uses representative spreadsheets and PDFs to check matching, preserved descriptions, readable output, duplicate-tracking feedback, and finalization. Exceptions include missing columns, unmatched records, and repeated requests. Implementation review follows one batch from source files to final records and distinguishes preparation from stamping and finalization.

Build 4 Parcel Processing and Inventory Synchronization

Design and development connect packing activity and stock movements with product and parcel records. The current Deliveries station is documentary:

staff scan or type a tracking number, capture or upload a photo, and save it for later lookup. Saving the photo is separate from stock deduction.

Testing checks the designated packing-confirmation workflow and its stock source. Photo scenarios cover tracking numbers with and without parcel records, duplicates, and later RTS lookup. Saving a photo alone must leave product and packing-material balances unchanged. Implementation review distinguishes packing, shipment-status actions, and documentary saves.

Build 5 Point of Sale System

Design and development provide cashier PIN access, item selection or scanning, normal or RTS stock selection, checkout, receipts, and history. Receipt items, prices, totals, and consumed inventory remain connected. Discount and receivable operations follow their configured permissions.

Testing covers normal-stock sales, RTS sales, totals, insufficient stock, unauthorized discounts, and repeated checkout requests. Each test records initial stock and the expected deduction. Implementation review demonstrates a complete sale and retrieves its receipt and history.

Build 6 Return-to-Sender Management

Design and development separate case documentation from sellable stock restoration. Staff locate the parcel, record condition and evidence, then scan

accepted sellable goods into an owned draft session. Confirmation applies the reviewed stock-in; damaged goods remain outside the sellable balance.

Testing checks that case saving does not credit sellable stock, cancellation leaves balances unchanged, and confirmation posts the expected quantity once. Further scenarios cover invalid codes, repeated scans, mixed products, and later RTS sales. Implementation review traces a return from the case to confirmed stock while preserving its original history.

Planning and Conception Initiation Phase

Business Model Canvas

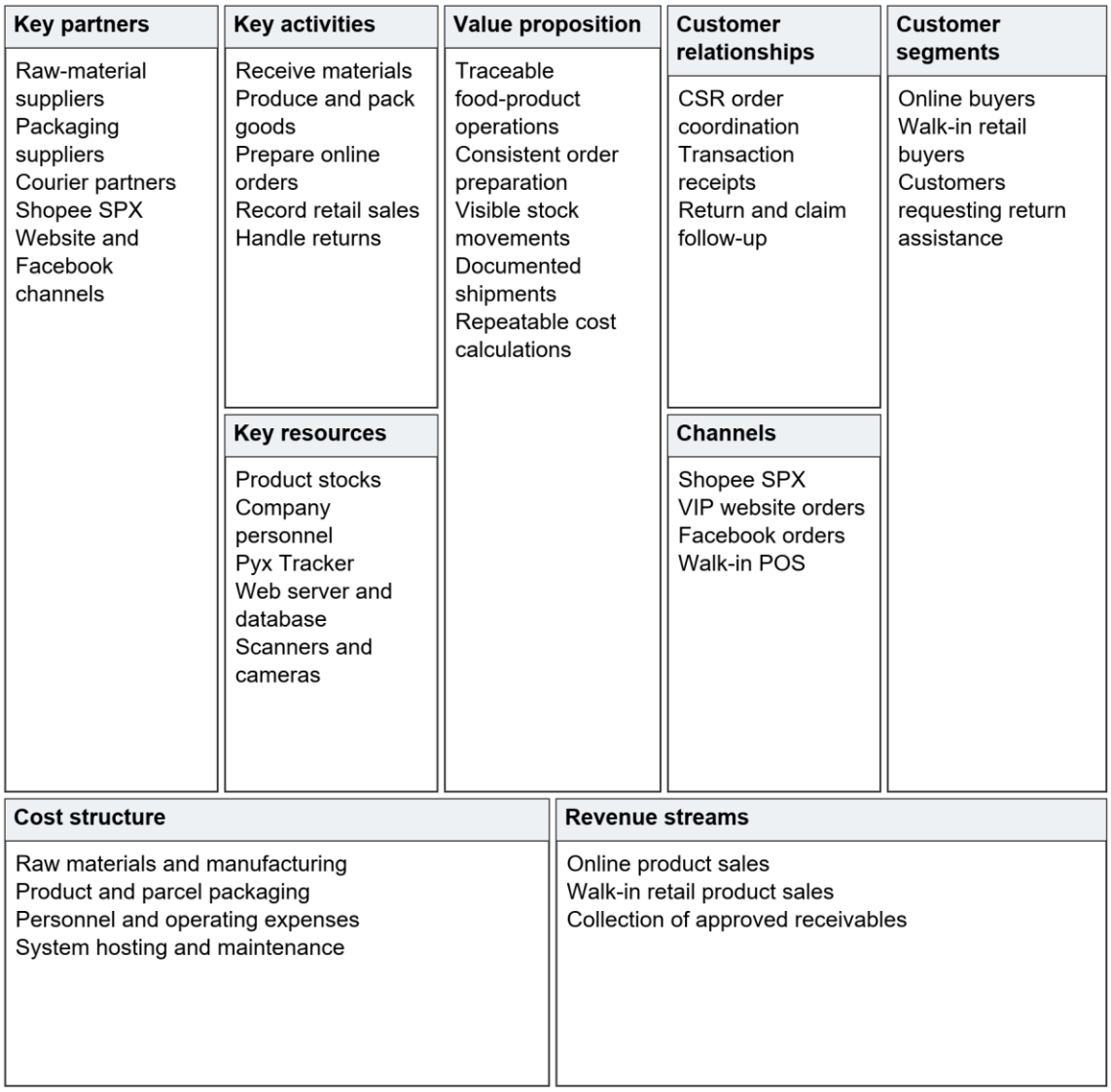


Figure 3.2 Business Model Canvas for Pyx Food Products and Pyx Tracker

The canvas follows the sample's nine blocks: partners, activities, resources, value proposition, customer relationships, channels, customer segments, cost

structure, and revenue streams. It represents the food business supported by the system. Pyx Tracker is an internal resource; the canvas does not invent software-subscription income, market-size figures, or financial amounts.

Gantt Chart

System Development Schedule | May–August

Features and activities	May				June				July				August			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Feature 1 — Integrated Inventory Management																
Design and Development																
Testing																
Implementation																
Feature 2 — Product Costing and Pricing																
Design and Development																
Testing																
Implementation																
Feature 3 — Multi-Platform Orders and Waybills																
Design and Development																
Testing																
Implementation																
Feature 4 — Parcel Processing and Inventory Sync																
Design and Development																
Testing																
Implementation																
Feature 5 — Point of Sale																
Design and Development																
Testing																
Implementation																
Feature 6 — Return-to-Sender Management																
Design and Development																
Testing																
Implementation																

Figure 3.3 Development and Implementation Gantt Chart (May–August)

System development starts in May and ends in August. The chart groups the six system features, each with Design and Development, Testing, and Implementation activities. Numbered columns divide each month into four planning intervals. Activity bars show a proposed allocation within the confirmed May–August period; exact milestone dates should be reconciled with the project records.

Functional Decomposition Diagram

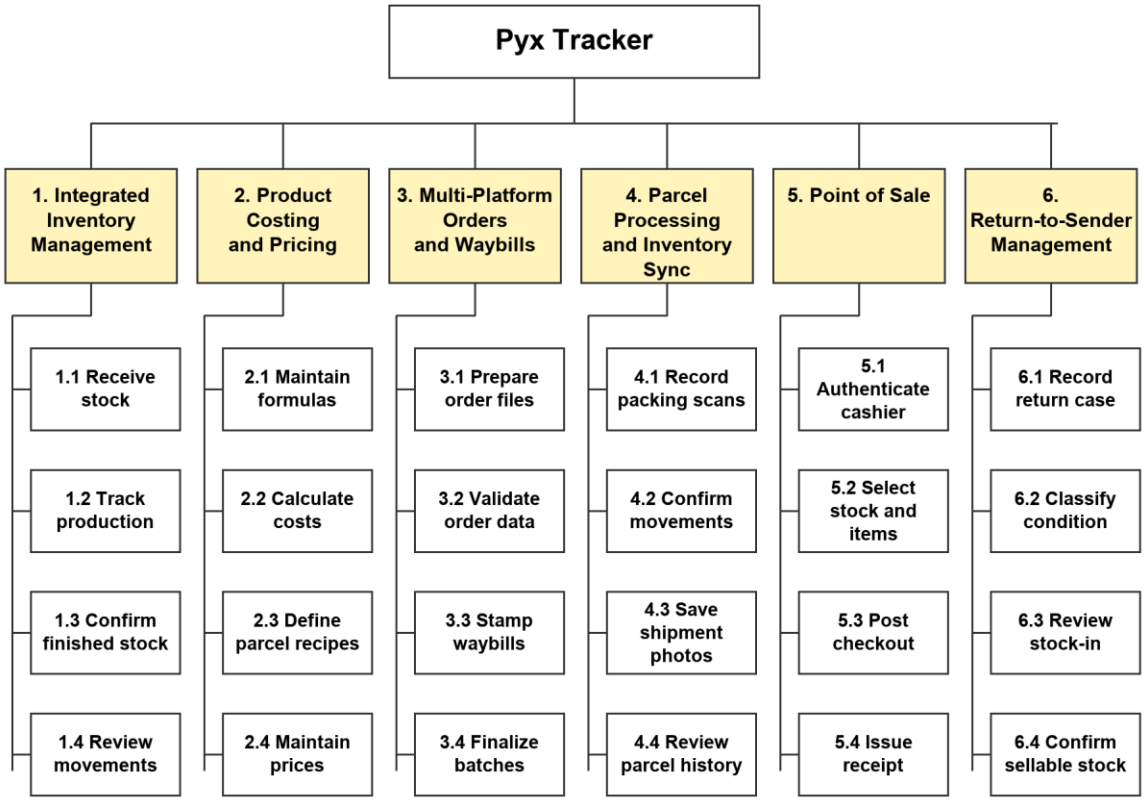


Figure 3.4 Functional Decomposition Diagram of Pyx Tracker

The FDD decomposes the root into six modules and their subfunctions, following the sample's system-to-function-to-subfunction hierarchy. It represents

responsibility, not screen navigation or execution order. Shared authentication, permissions, and reporting support the branches.

Analysis and Design Phase

Use Case Diagrams

The overall use case diagram places the operational actors outside one Pyx Tracker system boundary. Solid lines associate roles with their goals; dashed «include» arrows point to required behavior, while the optional approved discount «extend» arrow points to checkout. Production personnel also save shipment photos. Permissions remain subject to the role and department checks described in the use case matrices.

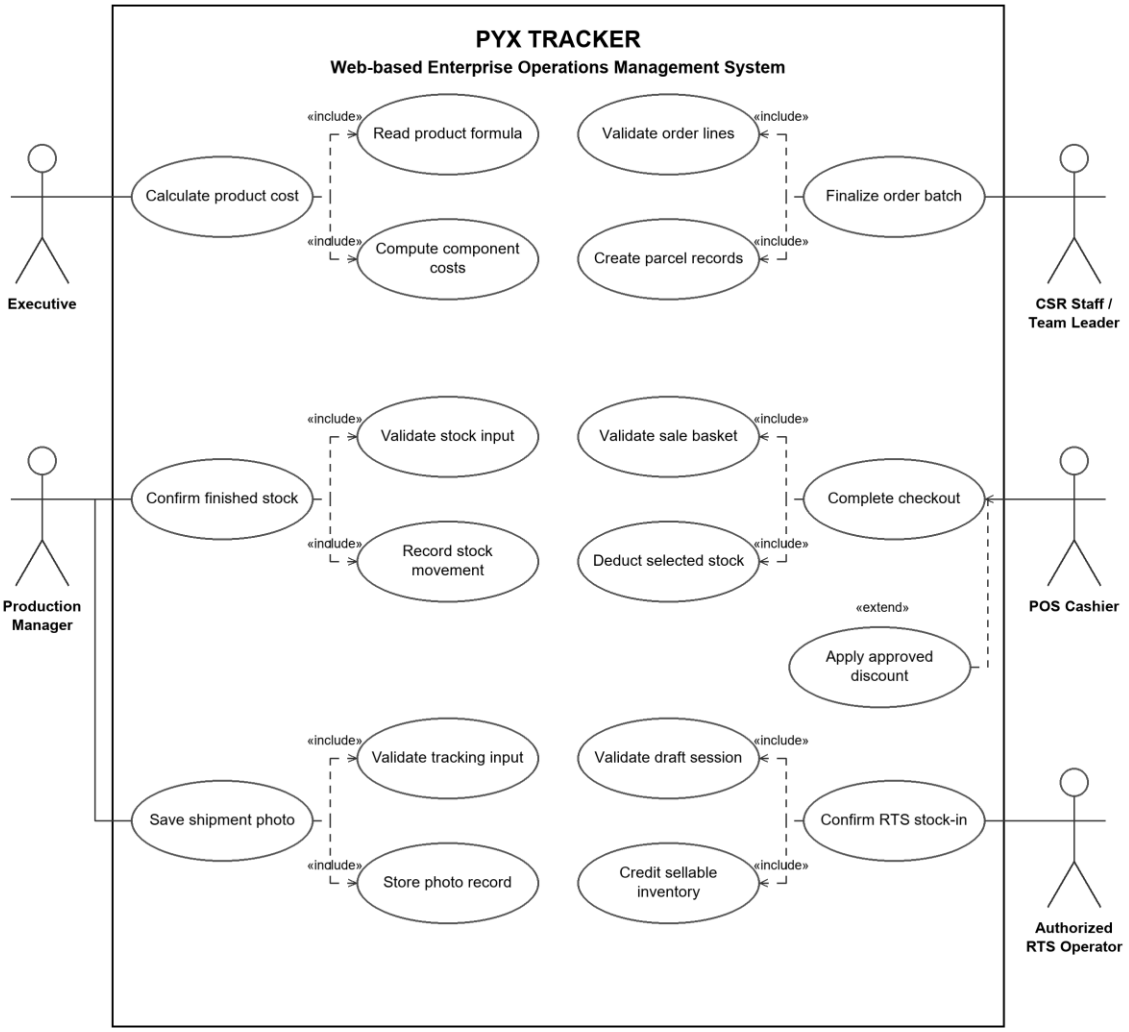


Figure 3.5 Overall Use Case Diagram of Pyx Tracker

Use Case Matrix

Table 3.1 Use Case Matrix for Inventory stock movement

Element	Specification
Use case	UC01 Inventory stock movement
Author	Cayunda, Klint Russel R.; Delos Santos, Brent R.; Fin, Noli III M.
Primary actor	Executive or authorized inventory operator
Trigger	Material receipt or finished-stock operation
Preconditions	Valid product, stock unit, permission and source record.
Basic flow	1. Select category and product. 2. Enter or scan the movement. 3. Review quantity and source. 4. Confirm the allowed operation.
Postcondition	The relevant balance and movement record reflect the confirmed quantity.
Exceptions	Reject invalid input; preserve balances when an unconfirmed action is cancelled.

Table 3.2 Use Case Matrix for Product and parcel costing

Element	Specification
Use case	UC02 Product and parcel costing
Author	Cayunda, Klint Russel R.; Delos Santos, Brent R.; Fin, Noli III M.
Primary actor	Executive
Trigger	Review or update of product costs
Preconditions	Configured material prices, units and formulas.
Basic flow	1. Select product or tier. 2. Review quantities, prices and yield. 3. Maintain formula lines. 4. Review calculated cost and price.

Element	Specification
Postcondition	The cost is reproducible from the stored inputs.
Exceptions	Deny non-executive access; identify missing configuration before relying on the result.

Table 3.3 Use Case Matrix for Order preparation and finalization

Element	Specification
Use case	UC03 Order preparation and finalization
Author	Cayunda, Klint Russel R.; Delos Santos, Brent R.; Fin, Noli III M.
Primary actor	Authorized CSR staff or team leader
Trigger	Receipt of supported order files
Preconditions	Supported source files and an active processing batch.
Basic flow	1. Choose preparation tool. 2. Upload and review data. 3. Run separate PDF/Excel stamper. 4. Download results and finalize batch.
Postcondition	Prepared outputs and finalized order/parcel records are available.
Exceptions	Report duplicates, unmatched records or missing fields; do not describe partial preparation as finalization.

Table 3.4 Use Case Matrix for Before-shipment documentation

Element	Specification
Use case	UC04 Before-shipment documentation
Author	Cayunda, Klint Russel R.; Delos Santos, Brent R.; Fin, Noli III M.
Primary actor	Executive or production manager
Trigger	Parcel presented for photo recording
Preconditions	Readable or manually entered tracking identifier and an image.

Element	Specification
Basic flow	1. Scan or type tracking. 2. Capture or upload photo. 3. Review and save. 4. Continue to next record.
Postcondition	Photo is retrievable by tracking number; stock is unchanged.
Exceptions	Provide duplicate feedback and request valid image/tracking input. A CSR parcel is not required.

Table 3.5 Use Case Matrix for Retail checkout

Element	Specification
Use case	UC05 Retail checkout
Author	Cayunda, Klint Russel R.; Delos Santos, Brent R.; Fin, Noli III M.
Primary actor	POS cashier
Trigger	Walk-in customer purchase
Preconditions	Valid POS access, product prices and available selected stock.
Basic flow	1. Select or scan items. 2. Select normal or RTS stock. 3. Review total. 4. Confirm checkout and issue receipt.
Postcondition	Sale and receipt exist; the selected stock is deducted by the completed transaction.
Exceptions	Handle insufficient stock and invalid input; restrict discounts to the authorized workflow.

Table 3.6 Use Case Matrix for RTS case and sellable stock-in

Element	Specification
Use case	UC06 RTS case and sellable stock-in
Author	Cayunda, Klint Russel R.; Delos Santos, Brent R.; Fin, Noli III M.

Element	Specification
Primary actor	Authorized RTS operator
Trigger	Returned parcel received
Preconditions	Tracking or case information, item condition and readable product codes.
Basic flow	1. Record case and condition. 2. Open stock-in session. 3. Scan received sellable goods. 4. Review and confirm session.
Postcondition	Case history is retained; confirmed sellable quantity is credited once.
Exceptions	Case save alone adds no sellable stock; discarding a draft leaves inventory unchanged.

Storyboard

The following labeled wireframes explain the principal screens and interaction sequences. They are schematic drawings, not production screenshots. They follow the reference's screen-plus-explanation approach without copying its sample company's interface. Approved screenshots of the exact evaluated release can replace them if deployed-screen evidence is required.

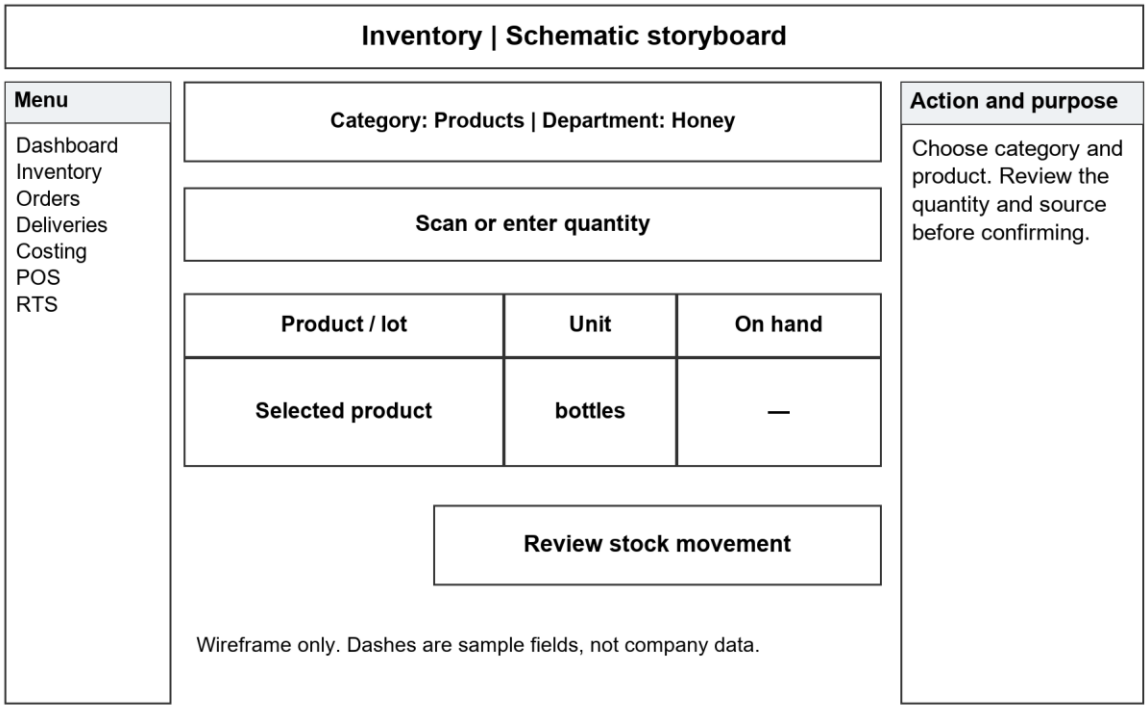


Figure 3.6 Inventory Storyboard

The user selects the category, product, and lot, enters or scans the movement, and reviews the action before confirmation. Quantities use the selected stock type's units. Available actions depend on the user's permissions.

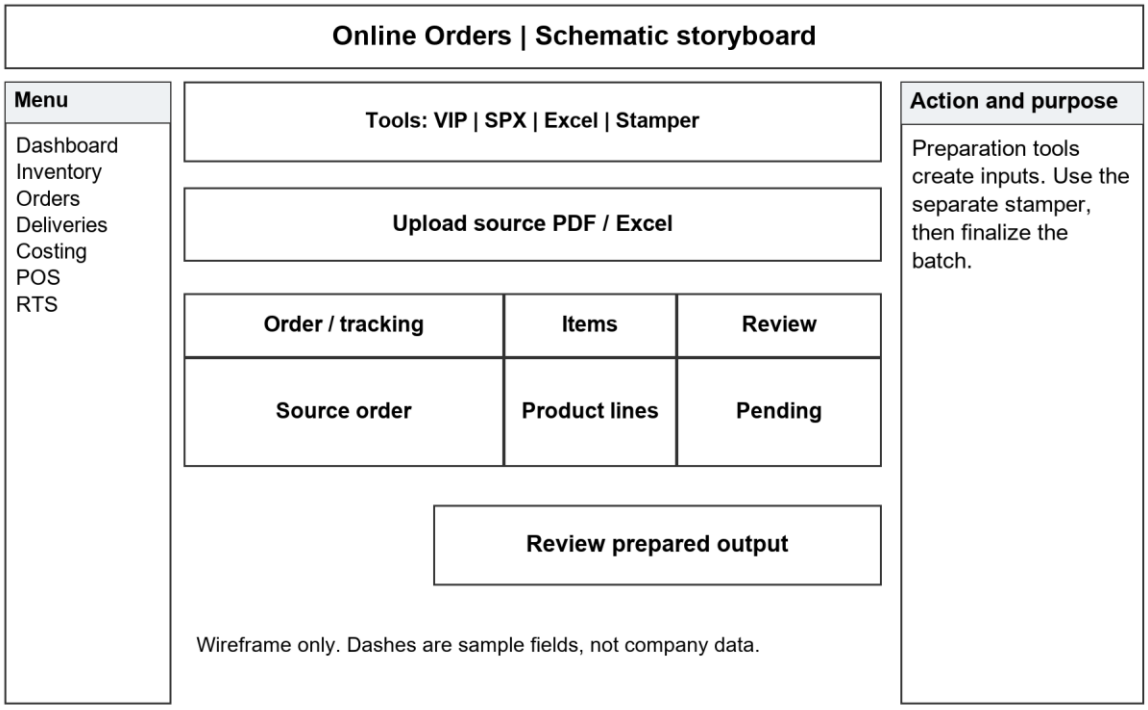


Figure 3.7 Online Orders Storyboard

CSR personnel select a source-specific preparation tool, review the information, then operate the separate stamper. Generated documents and finalized batch records are distinct outcomes.

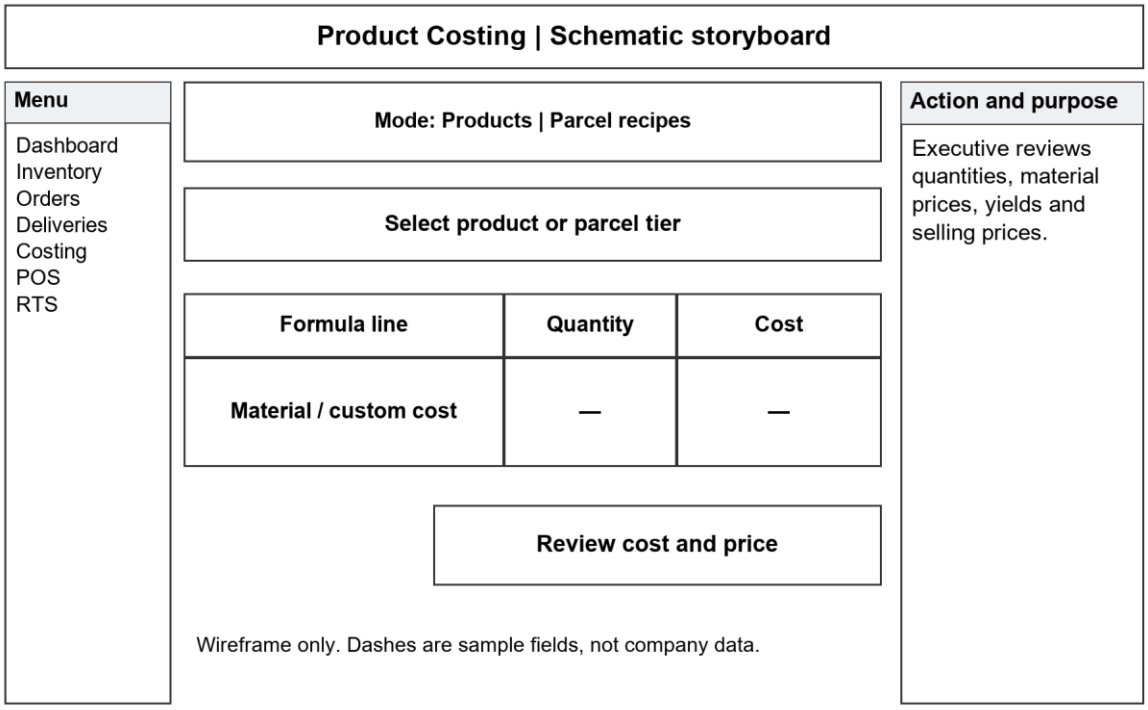


Figure 3.8 Product Costing Storyboard

An executive selects a product or parcel recipe, checks price and yield, and reviews formula or custom-cost lines. Product and parcel expenses remain separate.

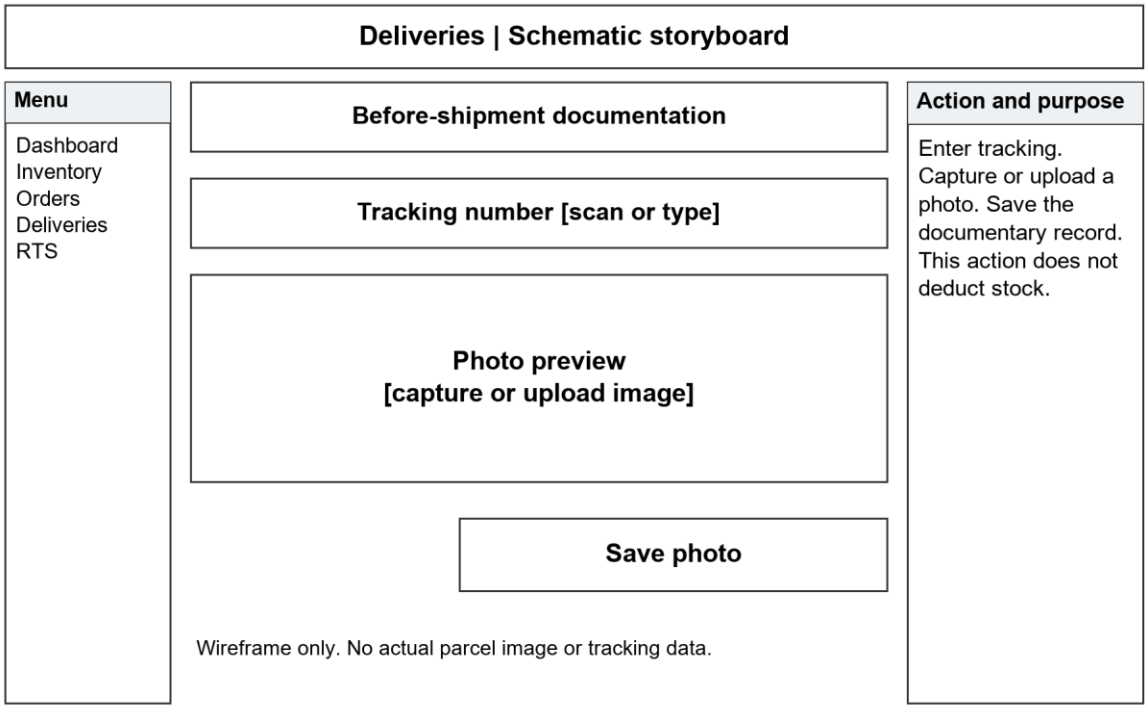


Figure 3.9 Parcel Documentation Storyboard

The operator enters the tracking number, captures or uploads the image, and saves the photo record. The storyboard does not depict stock deduction from this documentary action.

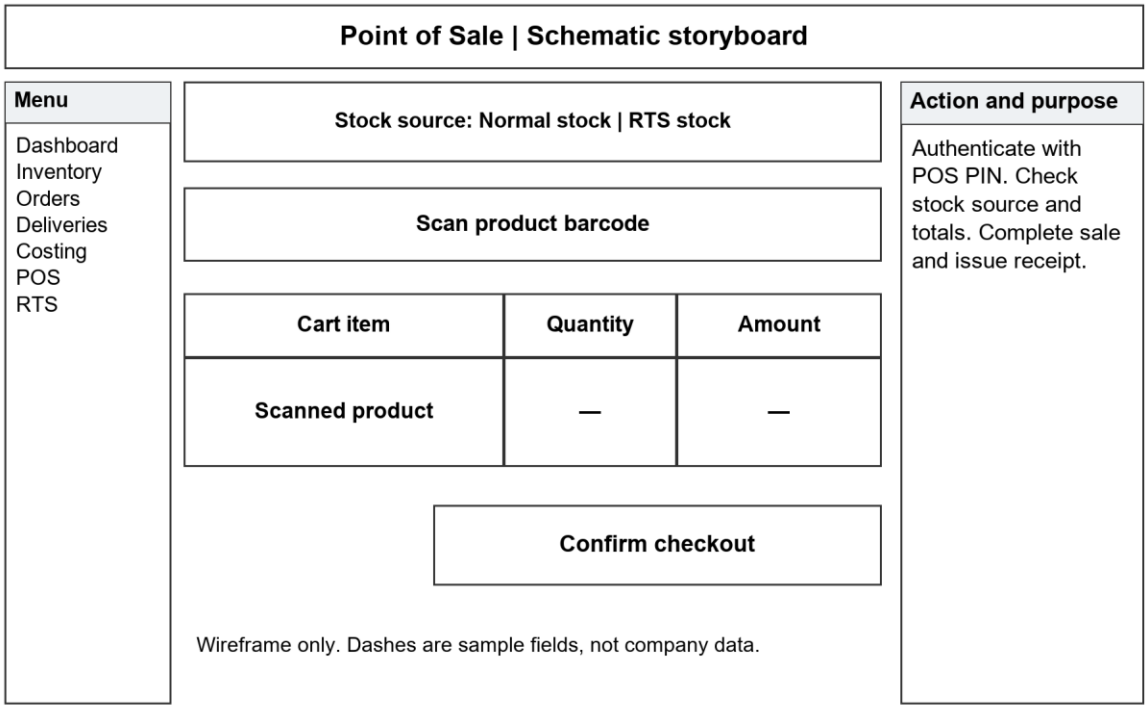


Figure 3.10 Point of Sale Storyboard

The cashier authenticates, chooses items and stock source, reviews the basket, and completes checkout. The receipt and ending stock provide evidence of the transaction.

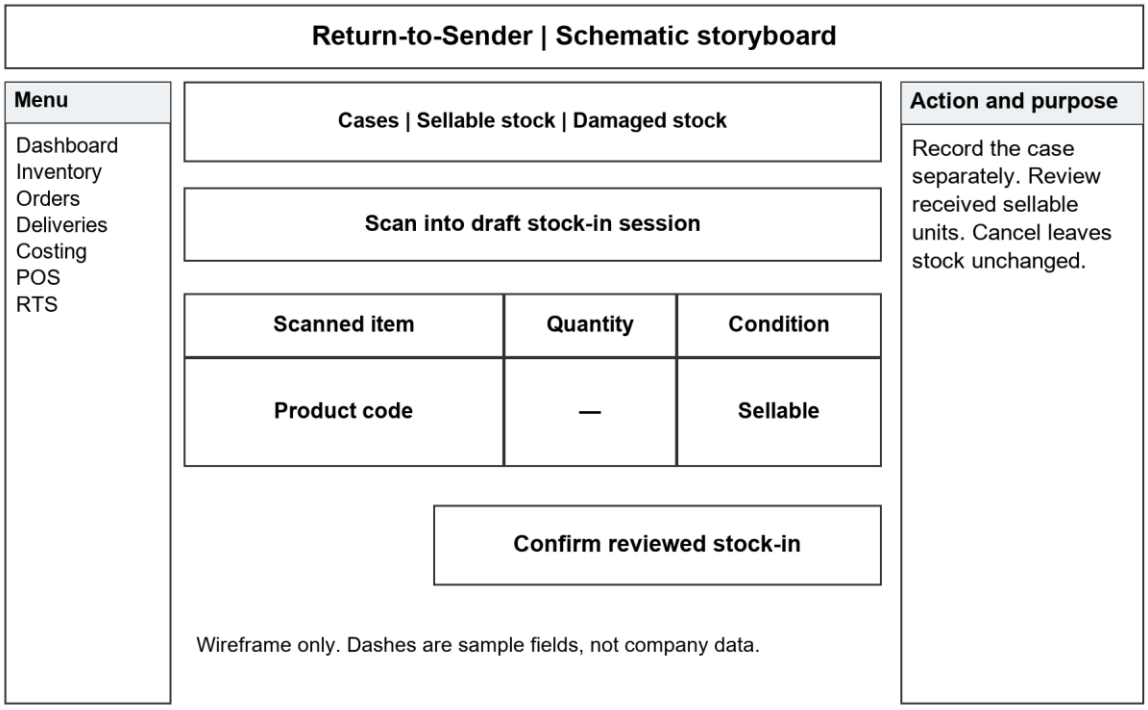


Figure 3.11 Return-to-Sender Storyboard

The operator records the case and condition, then reviews a separate stock-in session for received sellable units. Inventory is credited after confirmation.

Database Design

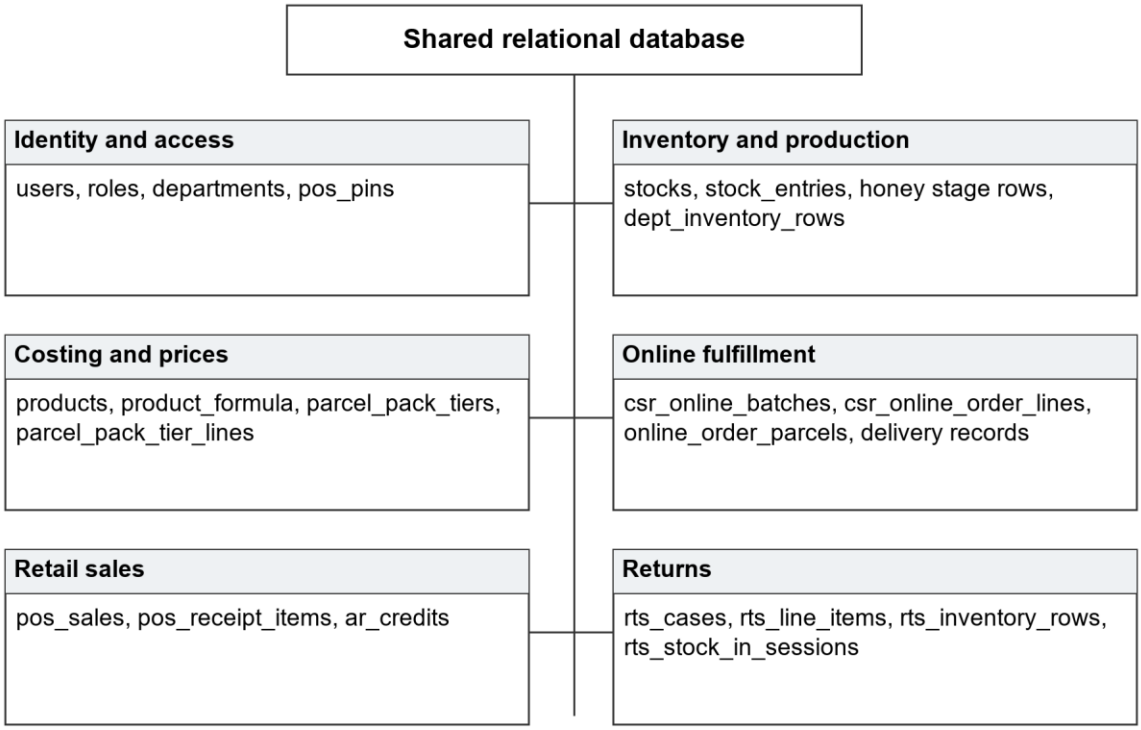


Figure 3.12 Database Design by Functional Area

The shared relational database contains identity, stock, production, order, costing, sales, and returns records. The areas shown are logical groupings, not independent databases. Dedicated tables preserve the different units and lifecycles of honey stages, department goods, material pools, and transactions.

Column and foreign-key metadata were inspected on 15 September 2026. Business records were not needed. The diagrams describe the reviewed local schema; deployment verification should confirm that the evaluated installation uses the same version.

Entity Relationship Diagram

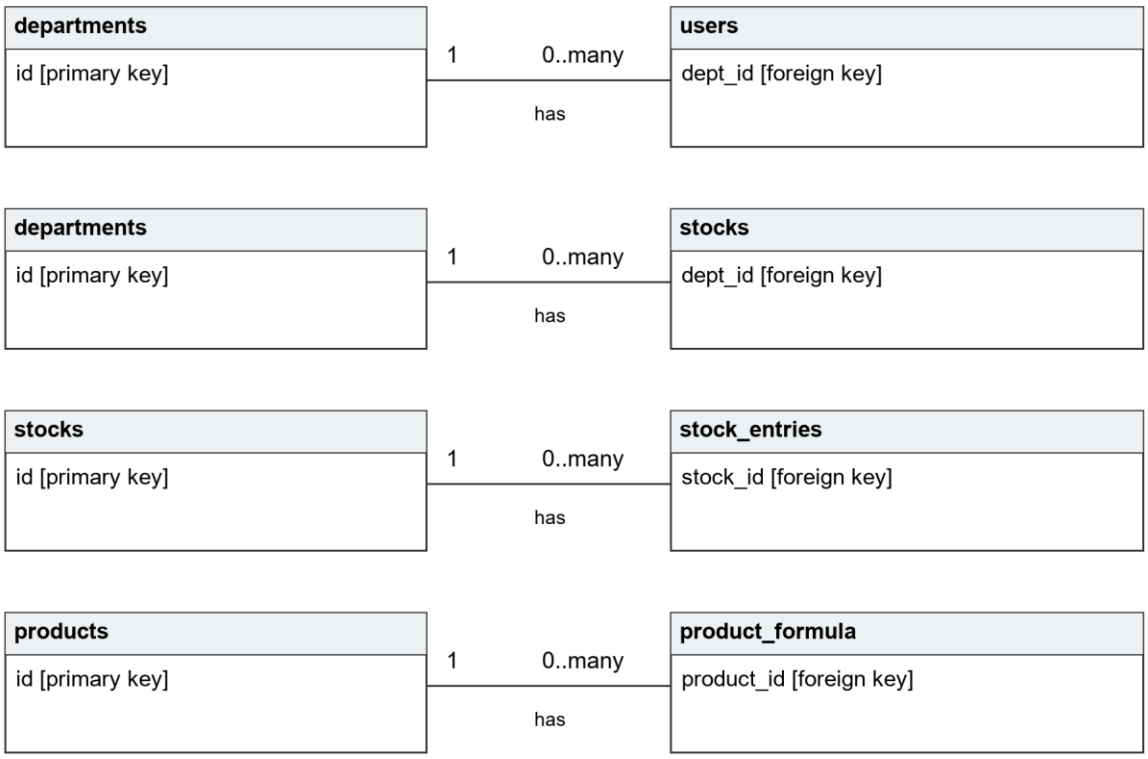


Figure 3.13 Inventory and Costing Entity Relationships

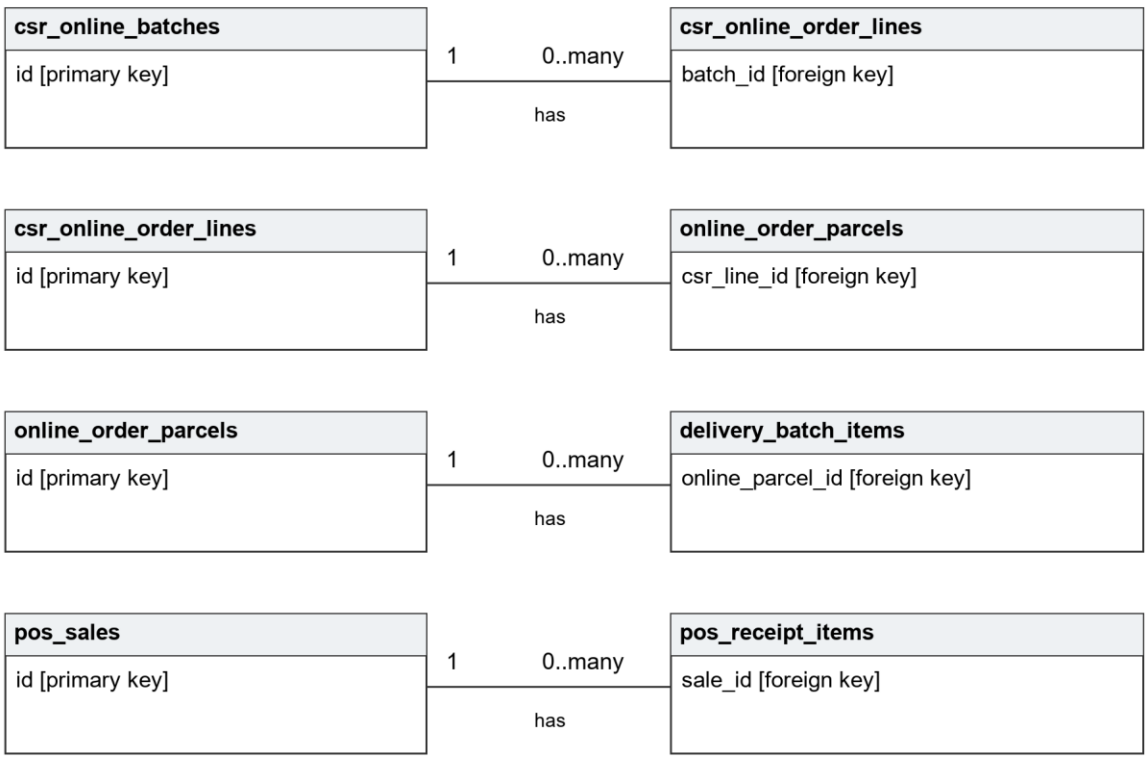


Figure 3.14 Order and Sales Entity Relationships

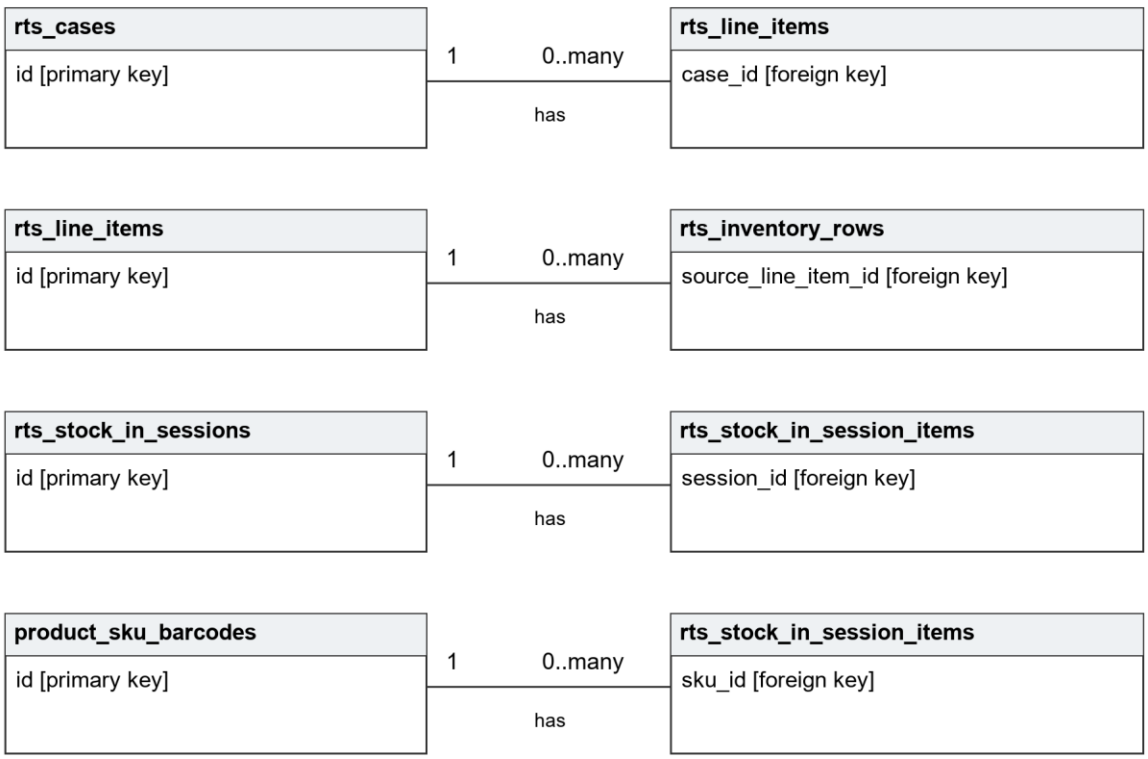


Figure 3.15 Return and Barcode Entity Relationships

These diagrams show selected declared foreign keys. A parent identifier may be referenced by many child records. The dictionary identifies whether a child reference permits NULL. Application-level relationships are not presented as database constraints. Repeated entity boxes keep each relationship readable.

Data Dictionary

The core dictionary records actual field names, types, nullability, defaults, and keys for the principal tables supporting the six modules. Descriptions explain operational purpose. A dash means no explicit default or a SQL NULL default

reported by metadata, not a literal stored dash. PRI means primary key, UNI means unique index, and MUL means a non-unique indexed column. Index labels do not by themselves establish foreign keys.

Supporting request, history, permission, issued-code, unit-tracking, and correction tables are not all reproduced in this core dictionary. Permanent SKU codes identify product types; additional unit and event tables preserve movement detail.

Table 3.7 Data Dictionary for users

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
username	varchar(64)	No	—	Username. Index: UNI.
name	varchar(120)	No	—	Name.
email	varchar(160)	Yes	NULL	Email. Index: UNI.
employee_id	varchar(24)	No	—	Employee id. Index: UNI.
password_hash	varchar(255)	No	—	Password verification hash.
role_id	int(10) unsigned	No	—	Role id. FK to roles.id. Index: MUL.
dept_id	int(10) unsigned	Yes	NULL	Dept id. FK to departments.id. Index: MUL.
is_active	tinyint(1)	No	1	Whether the record is enabled. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.
updated_at	datetime	No	current_timestamp()	Updated at. on update current_timestamp().

Table 3.8 Data Dictionary for roles

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
name	varchar(32)	No	—	Name. Index: UNI.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.9 Data Dictionary for departments

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
name	varchar(32)	No	—	Name. Index: UNI.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.10 Data Dictionary for suppliers

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
first_name	varchar(80)	No	—	First name.
last_name	varchar(80)	No	—	Last name. Index: MUL.
contact_number	varchar(32)	No	—	Contact number.
payment_account	varchar(80)	No	—	Payment account.
payment_account_number	varchar(64)	No	—	Payment account number.
address	varchar(255)	No	—	Address.
valid_id_filename	varchar(190)	No	—	Valid id filename.

Attribute	Type	Null	Default	Description
category	enum	No	—	Category. Index: MUL. Values: honey, paragis, black_rice, cacao, honey_comb.
is_active	tinyint(1)	No	1	Whether the record is enabled.
created_by	int(10) unsigned	No	—	Created by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.
updated_at	datetime	No	current_timestamp()	Updated at. on update current_timestamp().

Table 3.11 Data Dictionary for stocks

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
name	varchar(160)	No	—	Name.
dept_id	int(10) unsigned	No	—	Dept id. FK to departments.id. Index: MUL.
supplier_id	int(10) unsigned	Yes	NULL	Supplier id. FK to suppliers.id. Index: MUL.
unit	varchar(32)	No	'pcs'	Unit.
category	enum	No	'costing_only'	Category. Values: raw_material, packaging, supply, costing_only.
production_deduct	tinyint(1)	No	1	Whether configured production deduction applies.
pool_yield_qty	decimal(14,4)	Yes	NULL	Usable output per costing pool unit.
pool_yield_unit	varchar(32)	Yes	NULL	Pool yield unit.
on_hand	decimal(12,2)	No	0.00	Operational material balance.
is_active	tinyint(1)	No	1	Whether the record is enabled.
current_price	decimal(14,6)	No	0.000000	Current unit price used in costing.

Attribute	Type	Null	Default	Description
supplier_name	varchar(160)	Yes	NULL	Supplier name.
supplier_code	varchar(64)	Yes	NULL	Supplier code.
created_by	int(10) unsigned	No	—	Created by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.12 Data Dictionary for stock_entries

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
stock_id	int(10) unsigned	No	—	Stock id. FK to stocks.id. Index: MUL.
dept_id	int(10) unsigned	No	—	Dept id. FK to departments.id. Index: MUL.
qty	decimal(14,6)	No	—	Recorded quantity in the applicable unit.
batching_date	datetime	No	—	Batching date.
supplier_batch_no	varchar(120)	Yes	NULL	Supplier batch no.
entered_by	int(10) unsigned	No	—	Entered by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at. Index: MUL.

Table 3.13 Data Dictionary for honey_warehouse_rows

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
supplier_name	varchar(160)	No	—	Supplier name.
supplier_id	int(10) unsigned	Yes	NULL	Supplier id. FK to suppliers.id. Index: MUL.
supplier_code	varchar(32)	No	—	Supplier code. Index: MUL.

Attribute	Type	Null	Default	Description
arrival_date	date	No	—	Arrival date. Index: MUL.
variant	enum	No	—	Variant. Index: MUL. Values: light, amber, dark, other.
current_qty	decimal(12,2)	No	—	Current quantity in gallons.
initial_qty	decimal(12,2)	No	—	Initial quantity in gallons.
unit_cost_per_gallon	decimal(12,2)	Yes	NULL	Unit cost per gallon.
supplier_price_id	int(10) unsigned	Yes	NULL	Supplier price id. FK to supplier_honey_prices.id. Index: MUL.
cost_override	tinyint(1)	No	0	Cost override.
created_by	int(10) unsigned	No	—	Created by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.14 Data Dictionary for honey_production_rows

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
warehouse_row_id	int(10) unsigned	No	—	Warehouse row id. FK to honey_warehouse_rows.id. Index: MUL.
supplier_code	varchar(32)	No	—	Supplier code. Index: MUL.
arrival_date	date	No	—	Arrival date.
stock_out_date	date	Yes	NULL	Stock out date. Index: MUL.
daily_group_key	varchar(128)	Yes	NULL	Daily group key. Index: UNI.
variant	enum	No	—	Variant. Values: light, amber, dark, other.
current_qty	decimal(12,2)	No	—	Current quantity in gallons.
initial_qty	decimal(12,2)	No	—	Initial quantity in gallons.

Attribute	Type	Null	Default	Description
transferred_at	datetime	No	current_timestamp()	Transferred at.
created_by	int(10) unsigned	No	—	Created by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.15 Data Dictionary for honey_packaging_rows

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
production_row_id	int(10) unsigned	Yes	NULL	Production row id. FK to honey_production_rows.id. Index: MUL.
supplier_code	varchar(32)	No	—	Supplier code.
batch_no	varchar(48)	No	—	Batch no. Index: MUL.
bottling_date	date	No	—	Bottling date. Index: MUL.
variant	enum	No	—	Variant. Values: light, amber, dark, other.
current_liter	decimal(12,2)	No	0.00	Current count of 1 L bottles.
initial_liter	decimal(12,2)	No	0.00	Initial count of 1 L bottles.
current_250ml	decimal(12,2)	No	0.00	Current count of 250 mL bottles.
initial_250ml	decimal(12,2)	No	0.00	Initial count of 250 mL bottles.
current_350ml	decimal(12,2)	No	0.00	Current count of 350 mL bottles.
initial_350ml	decimal(12,2)	No	0.00	Initial count of 350 mL bottles.
current_gallon	decimal(12,2)	No	0.00	Current count of gallon units.
initial_gallon	decimal(12,2)	No	0.00	Initial count of gallon units.
created_by	int(10) unsigned	No	—	Created by. FK to users.id. Index: MUL.

Attribute	Type	Null	Default	Description
created_at	datetime	No	current_timestamp()	Created at.

Table 3.16 Data Dictionary for dept_inventory_rows

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
dept_id	int(10) unsigned	No	—	Dept id. FK to departments.id. Index: MUL.
product_key	varchar(32)	Yes	NULL	Product key.
product_label	varchar(120)	No	—	Product label.
current_qty	decimal(12,2)	No	—	Current quantity for the configured stock item.
initial_qty	decimal(12,2)	No	—	Initial quantity for the configured stock item.
source_note	varchar(255)	Yes	NULL	Source note.
last_stock_in_at	datetime	Yes	NULL	Last stock in at.
created_by	int(10) unsigned	Yes	NULL	Created by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.17 Data Dictionary for products

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
name	varchar(160)	No	—	Name. Index: MUL.
dept_product_key	varchar(32)	Yes	NULL	Dept product key. Index: UNI.
dept_id	int(10) unsigned	No	—	Dept id. FK to departments.id. Index: MUL.
size	enum	Yes	NULL	Size. Values: liter, 350ml, 250ml, gallon.

Attribute	Type	Null	Default	Description
variant	enum	Yes	NULL	Variant. Values: light, amber, dark.
selling_price	decimal(12,2)	No	0.00	Selling price.
baseline_cost	decimal(12,2)	No	0.00	Baseline cost.
created_by	int(10) unsigned	No	—	Created by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.18 Data Dictionary for product_formula

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
product_id	int(10) unsigned	No	—	Product id. FK to products.id. Index: MUL.
stock_id	int(10) unsigned	Yes	NULL	Stock id. Index: MUL.
line_label	varchar(160)	Yes	NULL	Description for a material or custom cost line.
qty_per_unit	decimal(16,8)	No	—	Formula quantity required per sellable unit.
price_at_creation	decimal(12,2)	No	—	Price at creation.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.19 Data Dictionary for parcel_pack_tiers

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
family	enum	No	—	Family. Index: MUL. Values: honey, dept_pt_brc, jar_cacao.
tier_key	varchar(64)	No	—	Tier key. Index: UNI.
label	varchar(128)	No	—	Label.

Attribute	Type	Null	Default	Description
match_rule	longtext	No	—	Configured parcel composition matching rule.
is_active	tinyint(1)	No	1	Whether the record is enabled.
sort_order	int(11)	No	0	Sort order.
created_at	datetime	No	current_timestamp()	Created at.
updated_at	datetime	No	current_timestamp()	Updated at. on update current_timestamp().

Table 3.20 Data Dictionary for parcel_pack_tier_lines

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
tier_id	int(10) unsigned	No	—	Tier id. FK to parcel_pack_tiers.id. Index: MUL.
stock_id	int(10) unsigned	Yes	NULL	Stock id. FK to stocks.id. Index: MUL.
line_label	varchar(128)	Yes	NULL	Description for a material or custom cost line.
qty_typical	decimal(12,4)	No	0.0000	Typical recipe quantity.
qty_min	decimal(12,4)	Yes	NULL	Qty min.
qty_max	decimal(12,4)	Yes	NULL	Qty max.
unit	varchar(32)	No	'pcs'	Unit.
sort_order	int(11)	No	0	Sort order.

Table 3.21 Data Dictionary for csr_online_batches

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
user_id	int(10) unsigned	No	—	User id. Index: MUL.

Attribute	Type	Null	Default	Description
status	enum	No	'active'	Current workflow state. Values: active, processed, cancelled.
created_at	datetime	No	current_timestamp()	Created at. Index: MUL.
completed_at	datetime	Yes	NULL	Completed at.
master_file_token	varchar(64)	Yes	NULL	Master file token.
master_filename	varchar(255)	Yes	NULL	Master filename.

Table 3.22 Data Dictionary for csr_online_order_lines

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
batch_id	int(10) unsigned	No	—	Batch id. FK to csr_online_batches.id. Index: MUL.
run_id	int(10) unsigned	Yes	NULL	Run id. FK to csr_online_batch_runs.id. Index: MUL.
order_no	varchar(64)	Yes	NULL	Order no.
recipient_name	varchar(255)	No	''	Recipient name.
items	text	No	—	Items.
courier	varchar(191)	No	''	Courier. Index: MUL.
tracking_number	varchar(96)	No	''	Tracking identifier used for lookup and matching. Index: MUL.
source_pipeline	enum	No	—	Source pipeline. Index: MUL. Values: partner, vip, shopee_spx, tiktok.
partner_seller	enum	Yes	NULL	Partner seller. Index: MUL. Values: ian, jono, joven.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.23 Data Dictionary for online_order_parcels

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
csr_batch_id	int(10) unsigned	Yes	NULL	Csr batch id. FK to csr_online_batches.id. Index: MUL.
csr_line_id	int(10) unsigned	Yes	NULL	Csr line id. FK to csr_online_order_lines.id. Index: MUL.
tracking_number	varchar(96)	No	—	Tracking identifier used for lookup and matching. Index: UNI.
items_raw	text	No	—	Original item description.
items_parsed_json	longtext	Yes	NULL	Parsed parcel item representation.
recipient_name	varchar(255)	No	''	Recipient name.
courier	varchar(191)	No	''	Courier. Index: MUL.
order_no	varchar(64)	Yes	NULL	Order no.
source_pipeline	enum	No	—	Source pipeline. Index: MUL. Values: partner, vip, shopee_spx, tiktok.
partner_seller	enum	Yes	NULL	Partner seller. Index: MUL. Values: ian, jono, joven.
status	enum	No	'packing'	Current workflow state. Index: MUL. Values: packing, shipped, on_hold, cancelled, returned, rts.
ship_date	date	Yes	NULL	Ship date. Index: MUL.
return_date	date	Yes	NULL	Return date.
shipped_by_user_id	int(10) unsigned	Yes	NULL	Shipped by user id. FK to users.id. Index: MUL.
deducted_at	datetime	Yes	NULL	Timestamp recording parcel deduction.
rts_case_id	int(10) unsigned	Yes	NULL	Rts case id. Index: MUL.
status_before_rts	enum	Yes	NULL	Status before rts. Values: packing, shipped, on_hold, cancelled, returned.

Attribute	Type	Null	Default	Description
rts_recorded_at	datetime	Yes	NULL	Rts recorded at.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.24 Data Dictionary for delivery_batches

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
submitted_by_user_id	int(10) unsigned	No	—	Submitted by user id. FK to app_users.id. Index: MUL.
delivery_date	date	No	—	Delivery date. Index: MUL.
submitted_at	datetime	No	current_timestamp()	Submitted at. Index: MUL.

Table 3.25 Data Dictionary for delivery_batch_items

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
batch_id	int(10) unsigned	No	—	Batch id. FK to delivery_batches.id. Index: MUL.
photo_filename	varchar(190)	No	—	Stored photo file reference.
tracking_number	varchar(96)	Yes	NULL	Tracking identifier used for lookup and matching. Index: MUL.
raw_qr_text	varchar(512)	Yes	NULL	Raw qr text.
item_status	enum	No	'needs_manual'	Item status. Index: MUL. Values: decoded_ok, needs_manual, ambiguous, resolved.
online_parcel_id	int(10) unsigned	Yes	NULL	Optional related parcel identifier. FK to online_order_parcels.id. Index: MUL.
voided_at	datetime	Yes	NULL	Voided at.

Attribute	Type	Null	Default	Description
resolved_at	datetime	Yes	NULL	Resolved at.
resolved_by_user_id	int(10) unsigned	Yes	NULL	Resolved by user id. FK to app_users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.26 Data Dictionary for pos_sales

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
receipt_no	varchar(32)	No	—	Receipt no. Index: UNI.
buyer_name	varchar(160)	No	'Walk-in'	Buyer name.
buyer_credentials	varchar(160)	Yes	NULL	Buyer credentials.
seller_name	varchar(160)	Yes	NULL	Seller name.
subtotal_amount	decimal(12,2)	No	0.00	Subtotal amount.
discount_percent	tinyint(3) unsigned	Yes	NULL	Discount percent.
discount_amount	decimal(12,2)	No	0.00	Discount amount.
total_amount	decimal(12,2)	No	0.00	Total amount.
is_ar	tinyint(1)	No	0	Is ar. Index: MUL.
sold_by_pin_id	int(10) unsigned	Yes	NULL	Sold by pin id. FK to pos_pins.id. Index: MUL.
sold_by_user_id	int(10) unsigned	Yes	NULL	Sold by user id. FK to users.id. Index: MUL.
sale_date	datetime	No	current_timestamp()	Sale date. Index: MUL.
refunded_at	datetime	Yes	NULL	Refunded at. Index: MUL.
refund_reason	varchar(255)	Yes	NULL	Refund reason.

Attribute	Type	Null	Default	Description
created_at	datetime	No	current_timestamp()	Created at.

Table 3.27 Data Dictionary for pos_receipt_items

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
sale_id	int(10) unsigned	No	—	Sale id. FK to pos_sales.id. Index: MUL.
product_id	int(10) unsigned	No	—	Product id. FK to products.id. Index: MUL.
qty	decimal(12,2)	No	—	Recorded quantity in the applicable unit.
unit_price	decimal(12,2)	No	—	Unit price.
list_unit_price	decimal(12,2)	Yes	NULL	List unit price.
line_total	decimal(12,2)	No	—	Line total.
discount_percent	tinyint(3) unsigned	Yes	NULL	Discount percent.
variant	varchar(32)	Yes	NULL	Variant.
size	varchar(16)	Yes	NULL	Size.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.28 Data Dictionary for rts_cases

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
tracking_number	varchar(96)	No	—	Tracking identifier used for lookup and matching. Index: UNI.
courier_detected	varchar(16)	Yes	NULL	Courier detected.

Attribute	Type	Null	Default	Description
courier_final	varchar(16)	No	—	Courier final.
shop	varchar(24)	No	—	Shop. Index: MUL.
vip_subsource	varchar(16)	Yes	NULL	Vip subsource.
date_delivered	date	Yes	NULL	Date delivered. Index: MUL.
date_opened	date	No	—	Date opened.
documentation_photo	varchar(190)	Yes	NULL	Documentation photo.
documentation_attachments	longtext	Yes	NULL	Documentation attachments.
damaged_reason	enum	Yes	NULL	Damaged reason. Index: MUL. Values: leak, item_replaced, packaging_replaced.
damaged_tag	varchar(128)	No	'no'	Damaged tag.
case_disposition	enum	No	'standard'	Case disposition. Index: MUL. Values: standard, swapped, leak.
claim_status	enum	No	'for_submission'	Claim status. Index: MUL. Values: for_submission, denied_claims, approved_claims, submitted_claims, on_process.
claim_status_note	varchar(512)	No	''	Claim status note.
delivery_match_item_id	int(10) unsigned	Yes	NULL	Delivery match item id. FK to delivery_batch_items.id. Index: MUL.
before_shipment_photo_filename	varchar(255)	Yes	NULL	Before shipment photo filename.
parcel_info_snapshot	text	Yes	NULL	Parcel info snapshot.
created_at	datetime	No	current_timestamp()	Created at.
updated_at	datetime	No	current_timestamp()	Updated at. on update current_timestamp().
voided_at	datetime	Yes	NULL	Voided at. Index: MUL.

Attribute	Type	Null	Default	Description
voided_by_user_id	int(10) unsigned	Yes	NULL	Voided by user id.

Table 3.29 Data Dictionary for rts_line_items

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
case_id	int(10) unsigned	No	—	Case id. FK to rts_cases.id. Index: MUL.
product_type	varchar(32)	No	—	Product type.
honey_size	varchar(16)	Yes	NULL	Honey size.
honey_variant	varchar(16)	Yes	NULL	Honey variant.
combo_product	varchar(32)	Yes	NULL	Combo product.
combo_quantity	decimal(10,2)	Yes	NULL	Combo quantity.
combo_honey_size	varchar(16)	Yes	NULL	Combo honey size.
combo_honey_variant	varchar(16)	Yes	NULL	Combo honey variant.
quantity	decimal(10,2)	No	—	Quantity.
is_bonus	tinyint(1)	No	0	Is bonus.
bonus_parent_id	int(10) unsigned	Yes	NULL	Bonus parent id. Index: MUL.
line_role	enum	No	'reusable'	Classification as reusable or damaged. Index: MUL. Values: reusable, damaged.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.30 Data Dictionary for rts_inventory_rows

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.

Attribute	Type	Null	Default	Description
source_line_item_id	int(10) unsigned	Yes	NULL	Originating return line, when applicable. FK to rts_line_items.id. Index: UNI.
category	enum	No	'honey'	Category. Index: MUL. Values: honey, other.
inventory_pool	enum	No	'sellable'	Separates sellable and damaged inventory. Index: MUL. Values: sellable, damaged.
product_label	varchar(120)	No	—	Product label.
variant	varchar(32)	Yes	NULL	Variant.
current_liter	decimal(12,2)	No	0.00	Current count of 1 L bottles.
initial_liter	decimal(12,2)	No	0.00	Initial count of 1 L bottles.
current_250ml	decimal(12,2)	No	0.00	Current count of 250 mL bottles.
initial_250ml	decimal(12,2)	No	0.00	Initial count of 250 mL bottles.
current_350ml	decimal(12,2)	No	0.00	Current count of 350 mL bottles.
initial_350ml	decimal(12,2)	No	0.00	Initial count of 350 mL bottles.
current_gallon	decimal(12,2)	No	0.00	Current count of gallon units.
initial_gallon	decimal(12,2)	No	0.00	Initial count of gallon units.
current_qty	decimal(12,2)	No	0.00	Current quantity for the configured stock item.
initial_qty	decimal(12,2)	No	0.00	Initial quantity for the configured stock item.
synced_at	datetime	No	current_timestamp()	Synced at.
created_at	datetime	No	current_timestamp()	Created at.

Table 3.31 Data Dictionary for rts_stock_in_sessions

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.

Attribute	Type	Null	Default	Description
started_by	int(10) unsigned	No	—	Started by. FK to users.id. Index: MUL.
status	enum	No	'draft'	Current workflow state. Values: draft, confirmed, discarded.
confirmed_at	datetime	Yes	NULL	Time the reviewed session was confirmed.
confirmed_by	int(10) unsigned	Yes	NULL	Confirmed by. FK to users.id. Index: MUL.
discarded_at	datetime	Yes	NULL	Time the draft session was discarded.
discarded_by	int(10) unsigned	Yes	NULL	Discarded by. FK to users.id. Index: MUL.
created_at	datetime	No	current_timestamp()	Created at.
updated_at	datetime	No	current_timestamp()	Updated at. on update current_timestamp().

Table 3.32 Data Dictionary for rts_stock_in_session_items

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
session_id	int(10) unsigned	No	—	Session id. FK to rts_stock_in_sessions.id. Index: MUL.
kind	enum	No	—	Kind. Values: honey, dept.
lookup_kind	enum	No	—	Whether the scanned code resolves as SKU or unique unit. Values: sku, unique_honey, unique_dept.
code	varchar(64)	No	—	Stored barcode identifier.
sku_id	int(10) unsigned	Yes	NULL	Sku id. FK to product_sku_barcodes.id. Index: MUL.
honey_barcode_id	int(10) unsigned	Yes	NULL	Honey barcode id. FK to honey_bottle_barcodes.id. Index: MUL.
dept_barcode_id	int(10) unsigned	Yes	NULL	Dept barcode id. FK to dept_pouch_barcodes.id. Index: MUL.

Attribute	Type	Null	Default	Description
variant	varchar(32)	Yes	NULL	Variant.
size	varchar(16)	Yes	NULL	Size.
product_key	varchar(32)	Yes	NULL	Product key.
product_label	varchar(96)	No	—	Product label.
qty	int(10) unsigned	No	1	Recorded quantity in the applicable unit.
created_at	datetime	No	current_timestamp()	Created at.
updated_at	datetime	No	current_timestamp()	Updated at. on update current_timestamp().

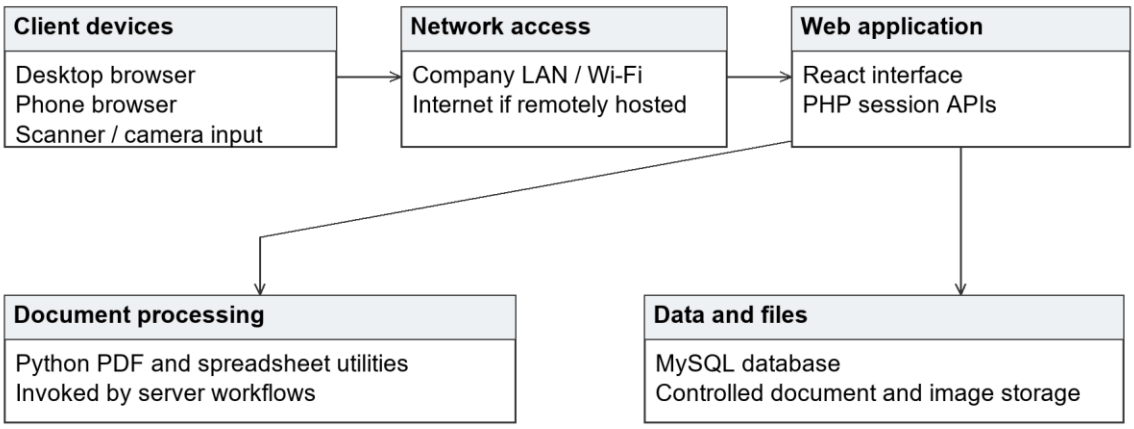
Table 3.33 Data Dictionary for product_sku_barcodes

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
kind	enum	No	—	Kind. Index: MUL. Values: honey, dept.
variant	varchar(32)	No	''	Variant.
size	varchar(16)	No	''	Size.
product_key	varchar(32)	No	''	Product key.
code	varchar(32)	No	—	Stored barcode identifier. Index: UNI.
updated_by	int(10) unsigned	Yes	NULL	Updated by. FK to users.id. Index: MUL.
updated_at	datetime	No	current_timestamp()	Updated at. on update current_timestamp().

Table 3.34 Data Dictionary for sku_barcode_stock_events

Attribute	Type	Null	Default	Description
id	int(10) unsigned	No	—	Row identifier. Index: PRI. auto_increment.
kind	enum	No	—	Kind. Values: honey, dept.
sku_id	int(10) unsigned	No	—	Sku id. FK to product_sku_barcodes.id. Index: MUL.
qty	int(10) unsigned	No	—	Recorded quantity in the applicable unit.
packaging_row_id	int(10) unsigned	Yes	NULL	Packaging row id.
honey_request_id	int(10) unsigned	Yes	NULL	Honey request id. Index: MUL.
dept_request_id	int(10) unsigned	Yes	NULL	Dept request id. Index: MUL.
scanned_by	int(10) unsigned	No	—	Scanned by. FK to users.id. Index: MUL.
scanned_at	datetime	No	current_timestamp()	Scanned at. Index: MUL.

Network Design



Logical deployment design. Exact hosting address, router model, and installed hardware require deployment records.

Figure 3.16 Logical Network Design of Pyx Tracker

Browser clients connect to the application server, PHP services, database, and controlled file storage. Server workflows invoke Python for document processing. Scanners and cameras provide input rather than connecting directly to the database. Session and permission checks apply through the application.

The diagram does not assert an unverified server address, hosting provider, router model, or equipment inventory. A local deployment requires company-network access; remote hosting also depends on an internet connection and appropriate secure browser access. Actual installation details require deployment records.

Network Topology

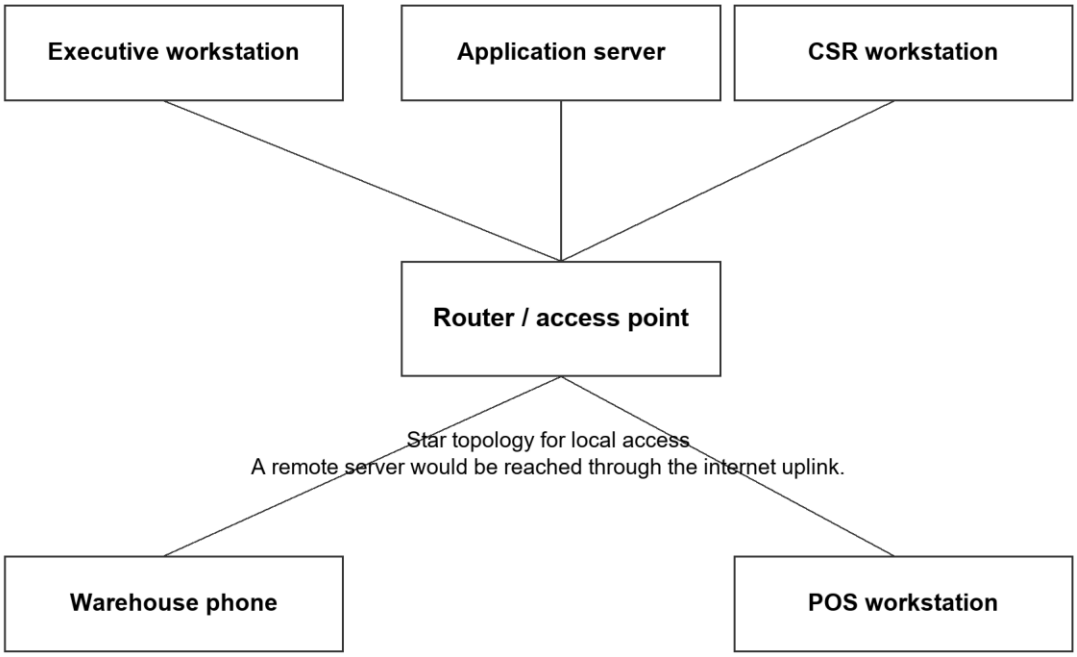


Figure 3.17 Star Topology for Local Access

Clients connect through a router or access point. A LAN server uses this local infrastructure; a remote server is reached through the internet uplink. This is a logical topology pending confirmation of the installed arrangement.

Technology Stack Development

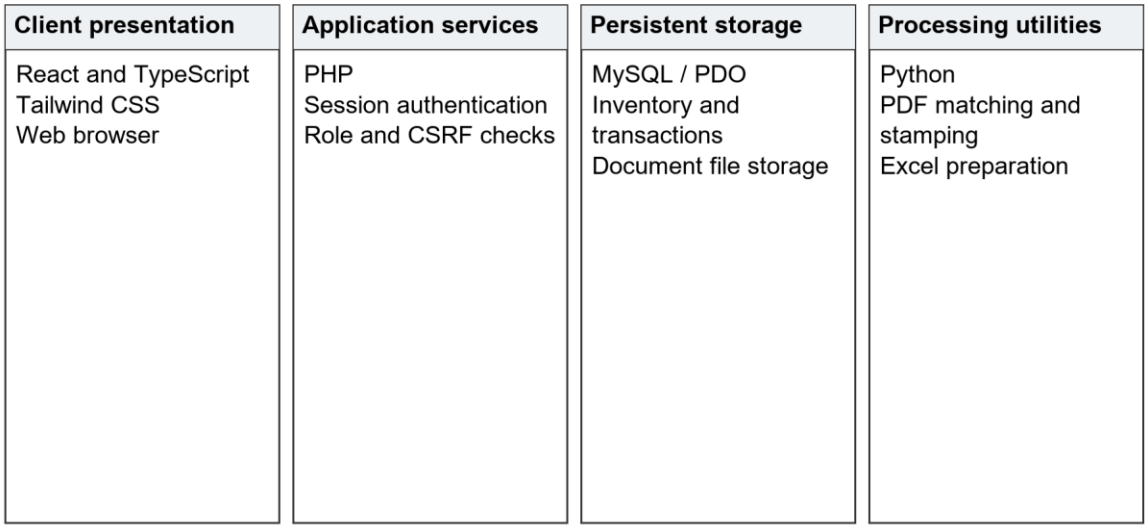


Figure 3.18 Technology Stack of Pyx Tracker

The stack follows the proposal and reviewed project: React and TypeScript for the interface, Tailwind CSS for presentation, PHP for business actions, MySQL for persistent records, and Python for document utilities. Vite supports frontend builds. Defined API actions connect these components.

Software Specification

Table 3.35 Software Specification

Component	Specification	Purpose
Client interface	React, TypeScript, Tailwind CSS	Browser screens and interactions.
Frontend build	Vite and project dependencies	Build the interface for deployment.

Component	Specification	Purpose
Server	PHP with session-based API actions	Validate access and execute business operations.
Database	MySQL accessed through PDO	Persist structured operational records.
Document utilities	Python and configured PDF/Excel libraries	Prepare and transform supported documents.
Development environment	Windows/XAMPP workspace; project editor	Local application development and maintenance.
Client browser	Compatible desktop or mobile browser	Access pages and supported scanning/camera workflows.

Hardware Specification

The sources do not identify exact deployed equipment. The table states functional requirements to be validated against the installed hardware rather than inventing machine specifications. Capacity must be tested against actual concurrent use and file sizes.

Table 3.36 Hardware Requirements to Confirm

Equipment	Functional requirement
Application server	Runs PHP, the database connection, and required Python utilities; storage sized for retained files and backups.
User workstation	Browser-capable computer with dependable network access.
Mobile device	Compatible browser and camera where camera scanning or photos are used.
Barcode scanner	Supported keyboard-wedge or equivalent input for the implemented scanning fields.
Printer	Receipt or barcode printer where printed output is used.

Equipment	Functional requirement
Network equipment	Router/access point and internet uplink where required by the hosting arrangement.
Backup storage	Separate capacity for the database and retained document/image files.

Program Specification

Table 3.37 Program Specification

Layer	Program responsibility
Presentation	React and TypeScript pages display role-appropriate workflows.
Authentication	PHP sessions, role/department checks, and POS authentication govern access.
Business processing	PHP handlers validate input, run the relevant stock or transaction action, and return a structured response.
Database access	PDO queries and transaction handling maintain persistent records.
File processing	Python utilities process supported PDF and Excel inputs; application storage controls their outputs.
Integrity controls	Validation, confirmation, duplicate checks, and history records protect the documented workflows.

Testing and Evaluation Procedure

Testing follows the six objectives. Each scenario records the initial state, permitted role, input, expected result, observed result, and evidence such as a receipt, generated file, or stock movement. Representative approved test records permit consistent comparisons.

Table 3.38 Functional Verification Plan

ID	Scenario	Expected evidence
T01	Inventory receipt and finished-stock confirmation	Correct units, movement record, and starting/ending balance.
T02	Product formula and parcel recipe calculation	Reproducible line values and totals; access denied to an unauthorized role.
T03	Prepare, stamp, and finalize an order batch	Correct generated files and records, with duplicate/unmatched input feedback.
T04	Packing and separate photo save	Expected packing movement; photo save alone leaves stock unchanged.
T05	POS checkout using normal and RTS stock	Correct receipt, sales entry, and deduction from the selected source.
T06	RTS case, draft cancellation, and confirmation	Case alone adds no sellable stock; cancelled draft has no effect; confirmation applies reviewed quantity once.

Integration tests follow receipt-to-finished-stock, source-file-to-parcel, POS-to-receipt, and return-to-confirmed-stock paths. Permission tests use authorized and unauthorized roles. Exceptions include invalid codes, duplicates, incomplete files, insufficient stock, interruption, and cancellation of drafts.

User evaluation should involve representatives of the actual operational roles performing defined tasks. Records should identify the selection method, number of participants, instrument, date, and system release. Any comparison of task time or stock accuracy must use comparable conditions and explain the measurements. No respondent count, rating, or improvement percentage is claimed without those records.

Deployment verification identifies the release date, installed schema, application version, roles, input devices, and backup and restoration procedure. An acceptance record should identify the evaluated installation. The detailed Gantt activity allocation and logical network diagrams must be reconciled with these records before final submission.

List of Modules

Table 3.39 List of Modules

Module	Functions	Obj.
Integrated Inventory Management System	Stock categories, production stages, finished-goods stock-in, movements and review.	1
Product Costing and Pricing Calculator	Product formulas, material prices and yields, parcel recipes and selling prices.	2
Multi-Platform Order Processing and Waybill Management	VIP/SPX preparation, validation, PDF/Excel stamping and batch finalization.	3
Parcel Processing and Inventory Synchronization	Packing records, applicable stock movements, parcel history and separate photo documentation.	4
Point of Sale System	PIN access, selected stock source, checkout, receipts and sales history.	5
Return-to-Sender Management	Return cases, condition records, reviewed barcode stock-in and sellable balances.	6

REFERENCES

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Principles behind the Agile Manifesto.* https://agilemanifesto.org/principles.html

Bhatti, M. A., & Bauirzhanovna, B. A. (2023). Impact of intelligent inventory system on improvement of reverse logistics: A case of Saudi manufacturing industry. *Operational Research in Engineering Sciences: Theory and Applications*, 6(1). https://oresta.org/menu-script/index.php/oresta/article/view/512

Chocholáč, J., Hruška, R., & Sommerauerová, D. (2022). Effectiveness of automatic identification technologies in shipment consolidation. *Proceedings of the 10th Carpathian Logistics Congress* (pp. 53–58). https://doi.org/10.37904/clc.2022.4565

Divina, J., Olan, A., Perez, N. C., Sarmiento, M., & Acepcion, R. (2025). Data-driven point-of-sale and inventory system for Pastil sa Tabi: Integrating sales forecasting algorithms and predictive analytics. *International Journal of Research and Innovation in Applied Science*, 10(10), 826–838. https://doi.org/10.51584/IJRIAS.2025.1010000066

Ercilla, L., Garcia, J., Gonzales, A., & Relano, K. (2024). A research proposal of inventory and POS system for Bida Mart in the City of San Jose Del Monte, Bulacan. *Ascendens Asia Singapore – Bestlink College of the Philippines Journal of Multidisciplinary Research*, 4(1). https://ojs.aaresearchindex.com/index.php/aasgbcpjmra/article/view/13558

Goedhart, J., Haijema, R., Akkerman, R., & de Leeuw, S. (2023). Replenishment and fulfilment decisions for stores in an omni-channel retail network. *European Journal of Operational Research*, 311(3), 1009–1022. https://doi.org/10.1016/j.ejor.2023.06.018

GS1 US. (2026). *Advancing intelligent data capture in modern logistics operations.* https://documents.gs1us.org/adobe/assets/deliver/urn:aaid:aem:389a6053-bb10-454c-ac16-1c19ea2950f9/GS1-US-Advancing-Intelligent-Data-Capture-in-Modern-Logistics-Operations.pdf

Kamnardsiri, T., Charoenkwan, P., Malang, C., & Wudhikarn, R. (2022). 1D barcode detection: Novel benchmark datasets and comprehensive comparison of deep convolutional neural network approaches. *Sensors*, 22(22), 8788. https://doi.org/10.3390/s22228788

Laping, A. K. T., Mendoza, C. A. R., Tiulentino, K. N. L., Villaraza, R. M., Bucu, G. C., & Ngo, J. K. (2023). Designing an inventory management system for Convenience Store X using design thinking approach. *Proceedings of the International Conference on Industrial Engineering and Operations Management* (pp. 1703–1715). https://ieomsociety.org/proceedings/2023manila/480.pdf

Lorzano, E. R. B., Roda, I. G. S., Paulo, S. B. T., & Gabriel, A. D. (2022). Food retail company web-based POS with inventory and stock management. *WCSE 2022 Spring Event Proceedings* (pp. 371–385). https://doi.org/10.18178/wcse.2022.04.045

Munifah, Henny, & Aqham, A. A. (2026). Digitalization of production cost calculations using the variable costing method in food industry SMEs. *Jurnal Ilmiah Sistem Informasi*, 5(1). https://doi.org/10.51903/ydwg6m74

Nanayakkara, P. R., Jayalath, M. M., Thibbotuwawa, A., & Perera, H. N. (2022). A circular reverse logistics framework for handling e-commerce returns. *Cleaner Logistics and Supply Chain*, 5, 100080. https://doi.org/10.1016/j.clscn.2022.100080

United Nations Department of Economic and Social Affairs. (n.d.-a). *Goal 8.* https://sdgs.un.org/goals/goal8

United Nations Department of Economic and Social Affairs. (n.d.-b). *Goal 9.* https://sdgs.un.org/goals/goal9

United Nations Department of Economic and Social Affairs. (n.d.-c). *Goal 12.* https://sdgs.un.org/goals/goal12